

绪论

- 什么是算法？
 - 由两个计算的例子归纳出算法的特点
- 什么是好的算法，如何定义度量的尺子？
 - 衡量不同问题规模下的时间成本
 - 实验法可行吗？
 - 抽象理想平台做虚拟实验，统计基本操作的次数
- 尺子上的刻度是什么？
 - 大O记号
 - 常数、对数、多项式、指数复杂度
- 如何利用尺子来测量算法的优劣？
 - 级数求和、递归追踪、递归方程、动态规划

绪论
计算：工具

绳索计算机及其算法

❖ 输入：任给直线 l 及其上一点 A

输出：经过 A 做 l 的一条垂线

❖ 算法（2000 B.C.，古埃及人）

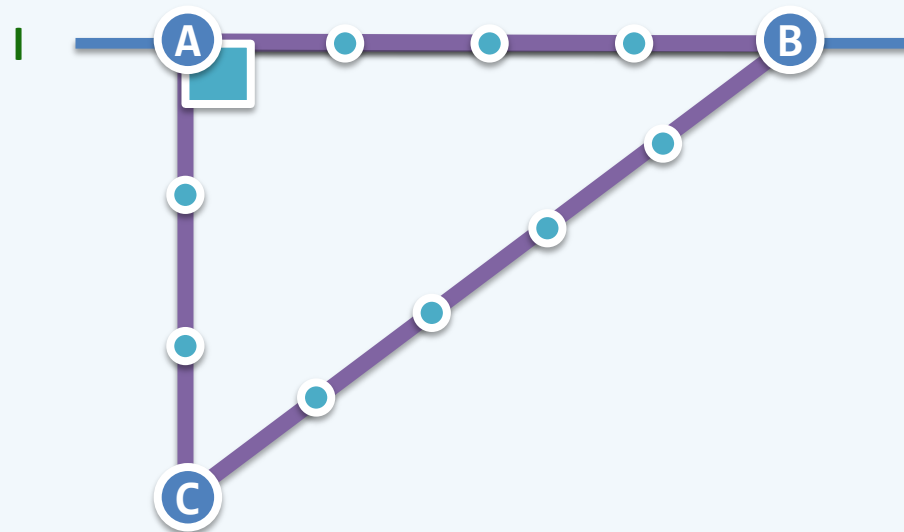
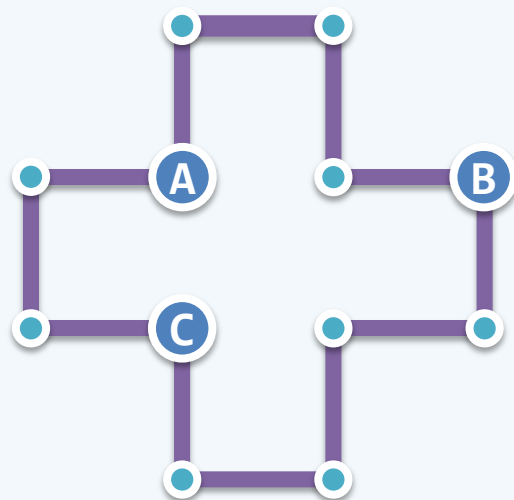
取12段等长的绳索，首尾联接成环

从 A 点起，将4段绳索沿 l 伸直并固定于 B

沿另一方向找到第3段绳索的终点 C

移动点 C ，将剩余的 $3 + 5$ 段绳索伸直

❖ 这里的计算机是什么？



尺规计算机

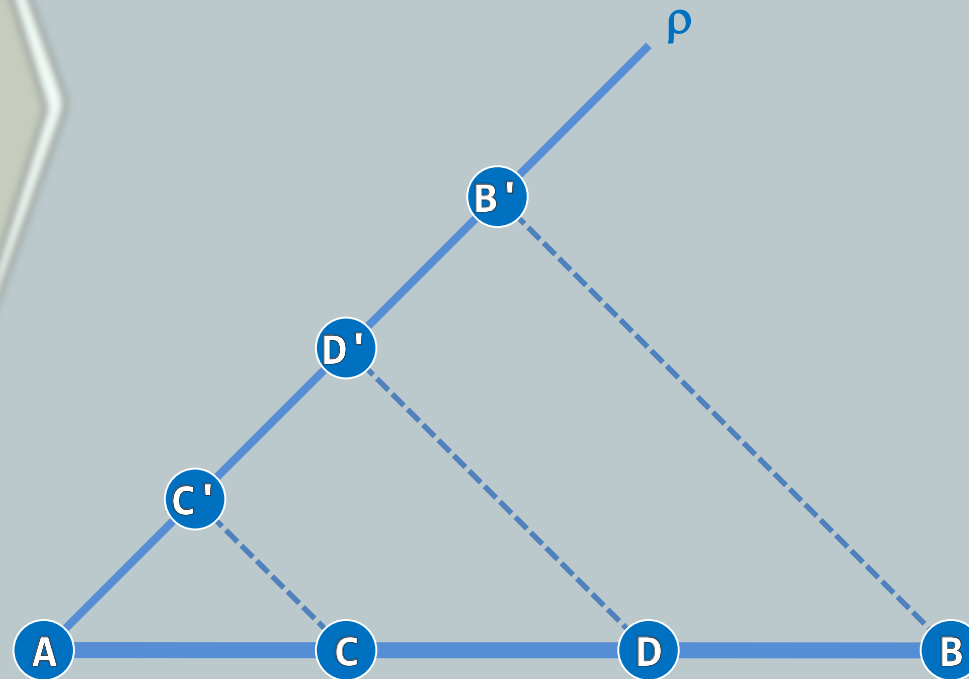
❖ 任给平面上线段 AB （输入），将其三等分（输出）

- 从 A 发出一条与 AB 不重合的射线 ρ
- 在 ρ 上取 $|AC'| = |C'D'| = |D'B'|$
- 联接 $B'B$
- 经 D' 做 $B'B$ 的平行线，交 AB 于 D
- 经 C' 做 $B'B$ 的平行线，交 AB 于 C

❖ 这里的计算机是什么？

❖ 它能够解决什么问题？不能解决什么问题？

❖ 子程序：过直线外一点，做平行线



绪论
计算： 算法

算法

❖ 计算 = 信息处理 = 借助某种工具，遵照一定规则，以明确而机械的形式进行

❖ 计算模型 = 计算机 = 信息处理工具

❖ 所谓算法，即特定计算模型下，旨在解决特定问题的指令序列

输入 待处理的信息（问题）

输出 经处理的信息（答案）

正确性 的确可以解决指定的问题

确定性 任一算法都可以描述为一个由基本操作组成的序列

可行性 每一基本操作都可实现，且在常数时间内完成

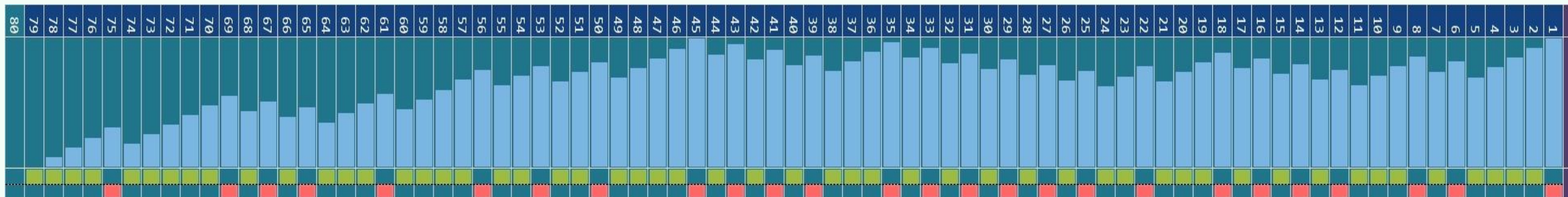
有穷性 对于任何输入，经有穷次基本操作，都可以得到输出

有穷性: Hailstone序列

$$Hailstone(n) = \begin{cases} \{1\} & (n \leq 1) \\ \{n\} \cup Hailstone(n/2) & (n \text{ is even}) \\ \{n\} \cup Hailstone(3n + 1) & (n \text{ is odd}) \end{cases}$$

42, 21, 64, 32, ..., 1

7, 22, 11, 34, 17, 52, 26, 13, 40, 20, 10, 5, 16, ..., 1



27, 82, 41, 124, 62, 31, 94, 47, 142, 71, 214, 107, ...

有穷性：程序 ~ 算法

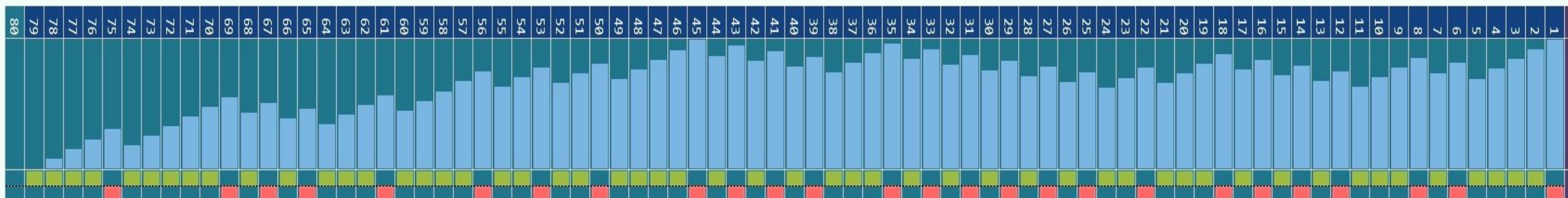
```
❖ int hailstone( int n ) {
```

```
    int length = 1;
```

```
    while ( 1 < n ) { n % 2 ? n = 3*n + 1 : n /= 2; length++; }
```

```
    return length;
```

```
} //对于任意的n, 总有|Hailstone(n)| < ∞ ?
```



❖ Erdos: Mathematics is **NOT** yet ready for such problems

好算法 = ?

❖ 正确：符合语法，能够编译、链接

能够正确处理**简单**的输入

能够正确处理**大规模**的输入

能够正确处理**一般性**的输入

能够正确处理**退化**的输入

能够正确处理**任意合法**的输入

❖ 健壮：能辨别不合法的输入并做适当处理

而不致非正常退出

❖ 可读：结构化 + 准确命名 + 注释 + ...

❖ 效率：速度尽可能快

存储空间尽可能少

Algorithms + Data Structures = Programs

(Algorithms + Data Structures) x Efficiency = Computation

绪论

计算模型：统一尺度

算法分析

❖ 两个主要方面...

❖ **正确**: 算法功能与问题要求一致?

数学证明? 可不那么简单...

❖ **成本**: 运行时间 + 所需存储空间

如何度量? 如何比较?

❖ 将计算成本描述为函数, 比如...

$T_A(P)$ = 算法A求解问题实例P的计算成本

❖ 意义不大, 毕竟...

可能出现的问题实例太多

❖ 如何归纳简化、概括?

❖ 观察: 问题实例的**规模**, 往往是

决定计算成本的**最主要**因素

❖ 通常: 规模接近, 计算成本也接近

规模扩大, 计算成本亦上升

特定算法 + 不同实例

❖ 令: $T_A(n)$ = 用算法A求解某一问题规模为n的实例, 所需的计算成本
讨论特定算法A (及其对应的问题) 时, 可简记作 $T(n)$

❖ 然而: 这一定义仍有问题...

❖ 观察: 同一问题等规模的不同实例, 计算成本毕竟不尽相同; 有些场合, 甚至会有实质差别...

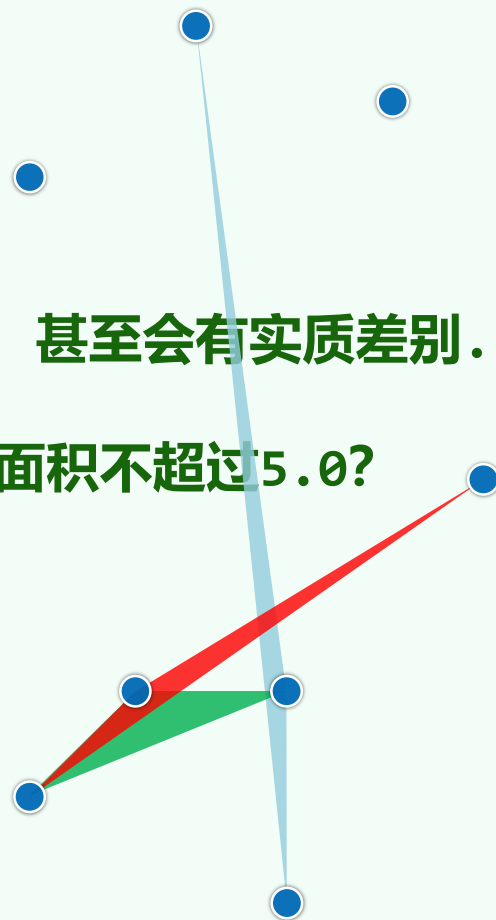
❖ 例如: 任给平面上n个点, 在它们定义的 $\binom{n}{3}$ 个三角形中, 是否某一个的面积不超过5.0?

❖ 蛮力: 最坏情况下需枚举所有三角形; 但运气好的话...

❖ 既然如此, 又该如何定义 $T(n)$ 呢?

❖ 稳妥起见, 取 $T(n) = \max\{ T(P) \mid |P| = n \}$

亦即, 在规模同为n的所有实例中, 只关注**最坏** (成本最高) 者



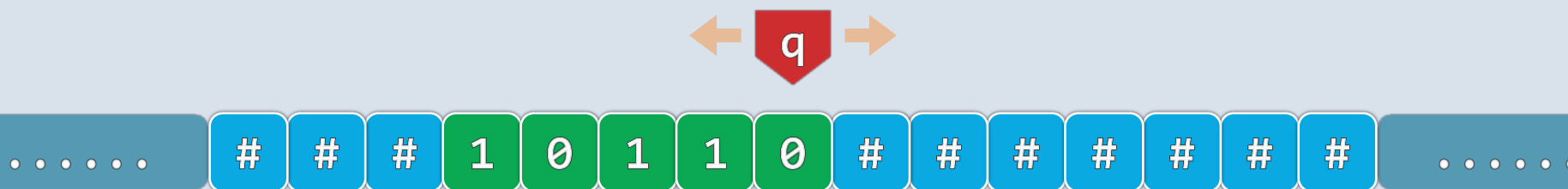
特定问题 + 不同算法

- ❖ 同一问题通常有多种算法，如何评判其优劣？
- ❖ 实验统计是最直接的方法，但足以准确反映算法的真正效率？ 不足够！
 - 不同的算法，可能更适应于不同**规模**的输入
 - 不同的算法，可能更适应于不同**类型**的输入
 - 同一算法，可能由不同**程序员**、用不同**程序语言**、经不同**编译器**生成
 - 同一算法，可能实现并运行于不同的**体系结构**、**操作系统**...
- ❖ 为给出**客观**的评判，需要抽象出一个**理想**的平台或模型
 - 不再依赖于上述种种具体的因素
 - 从而直接而准确地描述、测量并评价算法

绪论

计算模型：图灵机

构成部件



❖ Tape 依次均匀地划分为单元格

各存有某一字符，初始均为 '`#`'

❖ Head

- 总是对准某一单元格，并可读取或改写其中的字符
- 每经过一个节拍，可转向左侧或右侧的邻格

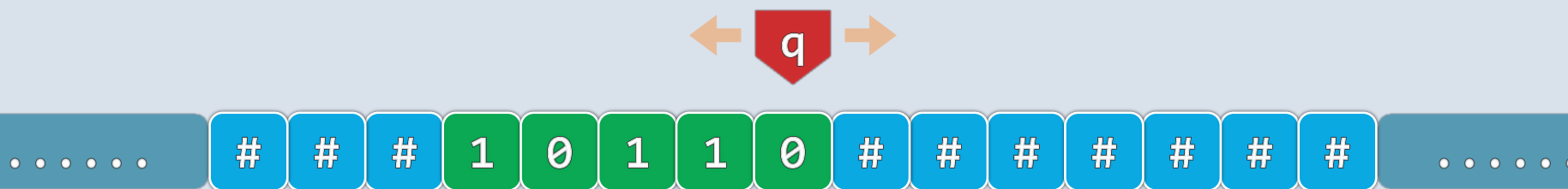
❖ Alphabet

- 字符的种类有限

❖ State

- TM总是处于有限种状态中的某一种
- 每经过一个节拍可按照规则转向另一种状态
- 统一约定， '`h`' = halt

转换函数



❖ Transition Function:

($q, c; d, L/R, p$)

❖ 特别地，一旦转入约定的状态 ' h '，则停机

❖ 从启动至停机，所经历的**节拍数目**

即可用以度量计算的**成本**

❖ 亦等于Head累计的**移动次数**（无量纲）

❖ 若当前状态为 q ，且当前字符为 c ，则

- 将当前字符改写为 d

- 转向左/右侧邻格

- 转入 ' p ' 状态

实例：Increase

❖ 功能：将二进制非负整数加一

❖ 原理：全'1'的后缀，翻转为全'0'

原最低位'0'或'#'翻转为'1'

❖ ($\leftarrow$, 1; 0, L, $\leftarrow$) //左行, 1 $\rightarrow$ 0

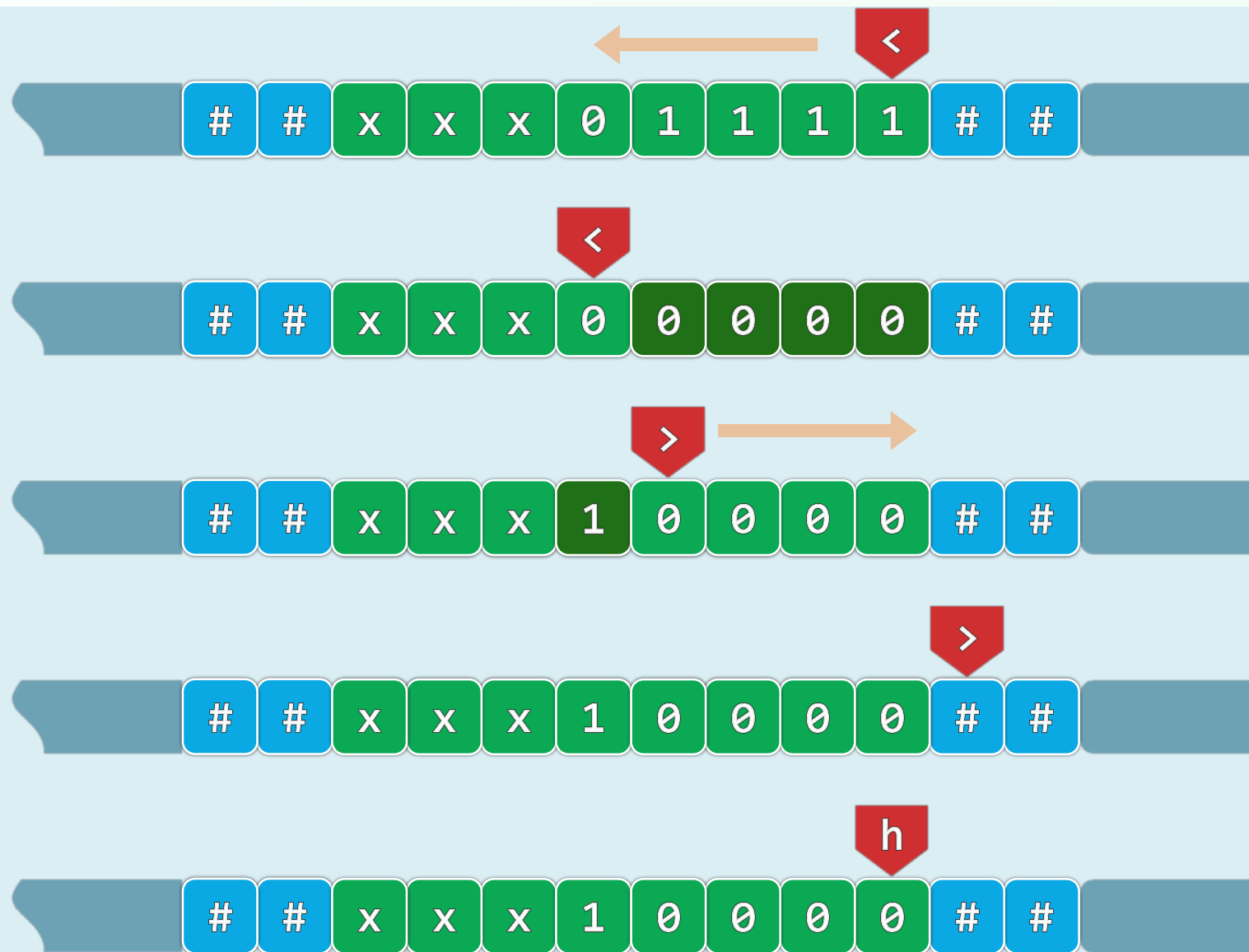
($\leftarrow$, 0; 1, R, $\rightarrow$) //掉头, 0 $\rightarrow$ 1

($\leftarrow$, #; 1, R, $\rightarrow$) //可否省略?

($\rightarrow$, 0; 0, R, $\rightarrow$) //右行

($\rightarrow$, #; #, L, h) //?

❖ 规范 ~ 接口



绪论

计算模型： RAM

Random Access Machine: 组成 + 语言



- ❖ 寄存器顺序编号，总数没有限制
- ❖ 可通过编号直接访问任意寄存器
- ❖ 每一基本操作仅需常数时间

//但愿如此

//call-by-rank

//循环及子程序本身非基本操作

$R[i] \leftarrow c$

$R[i] \leftarrow R[R[j]]$

$R[i] \leftarrow R[j] + R[k]$

$R[i] \leftarrow R[j]$

$R[R[i]] \leftarrow R[j]$

$R[i] \leftarrow R[j] - R[k]$

IF $R[i] = 0$ GOTO #

IF $R[i] > 0$ GOTO #

GOTO # STOP

Random Access Machine: 效率



- ❖ 与TM模型一样，RAM模型也是一般计算工具的简化与抽象
使我们可以**独立于**具体的平台，对算法的效率做出**可信**的比较与评判
- ❖ 在这些模型中
 - 算法的**运行时间** $\propto$ 算法需要执行的基本**操作次数**
 - $T(n)$ = 算法为求解规模为n的问题，所需执行的基本操作次数
- ❖ 思考：在TM、RAM等模型中衡量算法效率，为何通常只需考查运行时间？空间呢？

实例：Ceiling Division：思路

- $\spadesuit \quad \forall \quad c \geq 0 \quad \text{and} \quad d > 0$, define

$$\begin{aligned} \lceil c/d \rceil &= \min \{ x \mid c \leq d \cdot x \} \\ &= \min \{ x \mid c - 1 < d \cdot x \} \end{aligned}$$

❖ 例如: $\lceil 2/7 \rceil = 1$

$$\lceil 35/5 \rceil = 7 \qquad \lceil 2021/43 \rceil = 47$$

$$\lceil 41/7 \rceil = 6 \qquad \lceil 2022/43 \rceil = 48$$

❖ 思路：反复地从 $R[0] = c$ 中，减去 $R[1] = d$

统计在下溢之前，所做减法的次数x

[illegible]

实例: Ceiling Division: 算法

```
[0] R[3] <- 1 //constant increment
[1] GOTO 4 //in case R[0] = c == 0
[2] R[2] <- R[2] + R[3] //x++
[3] R[0] <- R[0] - R[1] //c -= d
[4] IF R[0] > 0 GOTO 2 //if c > 0 goto 2
[5] R[0] <- R[2] //else copy x to R[0] and
[6] STOP //return R[0] = x =  $\lceil c/d \rceil$ 
```

时间成本 ~ 各条指令执行次数之总和

Step	IR	R[0]	R[1]	R[2]	R[3]
0	[0]	25	12	0	0
1	[1]	^	^	^	1
2	[4]	^	^	^	^
3	[2]	^	^	^	^
4	[3]	^	^	1	^
5	[4]	13	^	^	^
6	[2]	^	^	^	^
7	[3]	^	^	2	^
8	[4]	1	^	^	^
9	[2]	^	^	^	^
10	[3]	^	^	3	^
11	[4]	-11	^	^	^
12	[5]	^	^	^	^
13	[6]	3	^	^	^

绪论

渐进复杂度：大O记号

渐近分析

❖ 回到原先的问题:

随着问题规模的增长, 计算成本如何增长?

❖ 这里更关心:

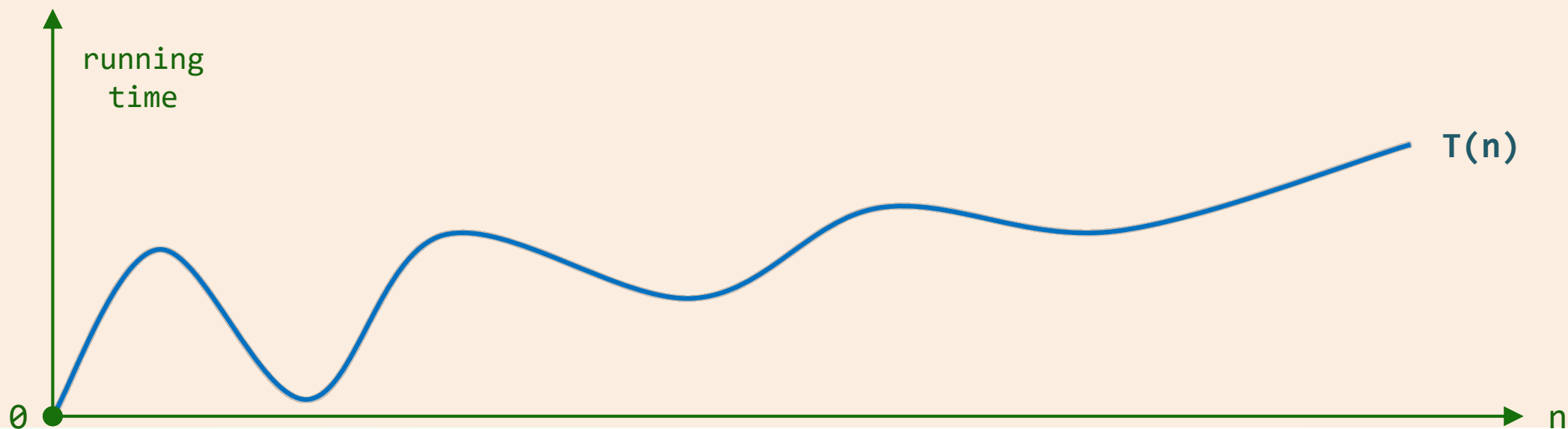
问题规模**足够大**之后, 计算成本的**增长趋势**

❖ 当输入规模 $n \gg 2$ 后, 算法

需执行的基本操作次数 $T(n) = ?$

❖ 如欲更为精确地估计, 还可考查

需占用的存储单元数 $S(n) = ?$



Big-O notation

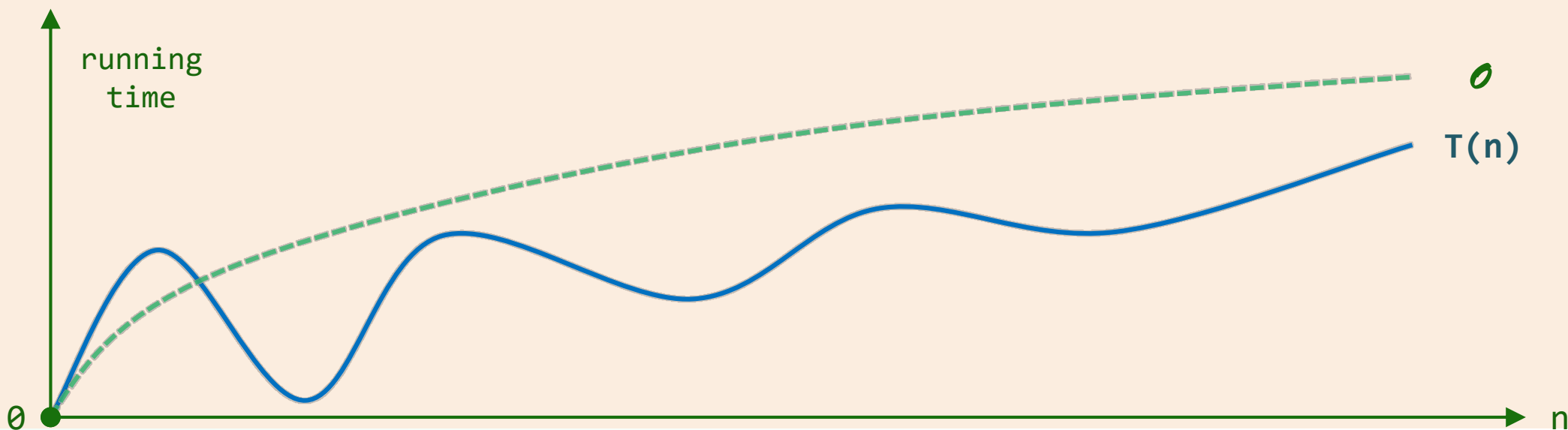
❖ **Paul Bachmann, 1894:** $T(n) = \mathcal{O}(f(n))$ iff $\exists c > 0$ s.t. $T(n) < c \cdot f(n) \quad \forall n \gg 2$

$$Ex: \sqrt{5n \cdot [3n \cdot (n+2) + 4] + 6} < \sqrt{5n \cdot [6n^2 + 4] + 6} < \sqrt{35n^3 + 6} < 6 \cdot n^{1.5} = \mathcal{O}(n^{1.5})$$

❖ 与 $T(n)$ 相比, $f(n)$ 在形式上更为简洁, 但依然反映前者的增长趋势

$$\mathcal{O}(f(n)) = \mathcal{O}(c \cdot f(n))$$

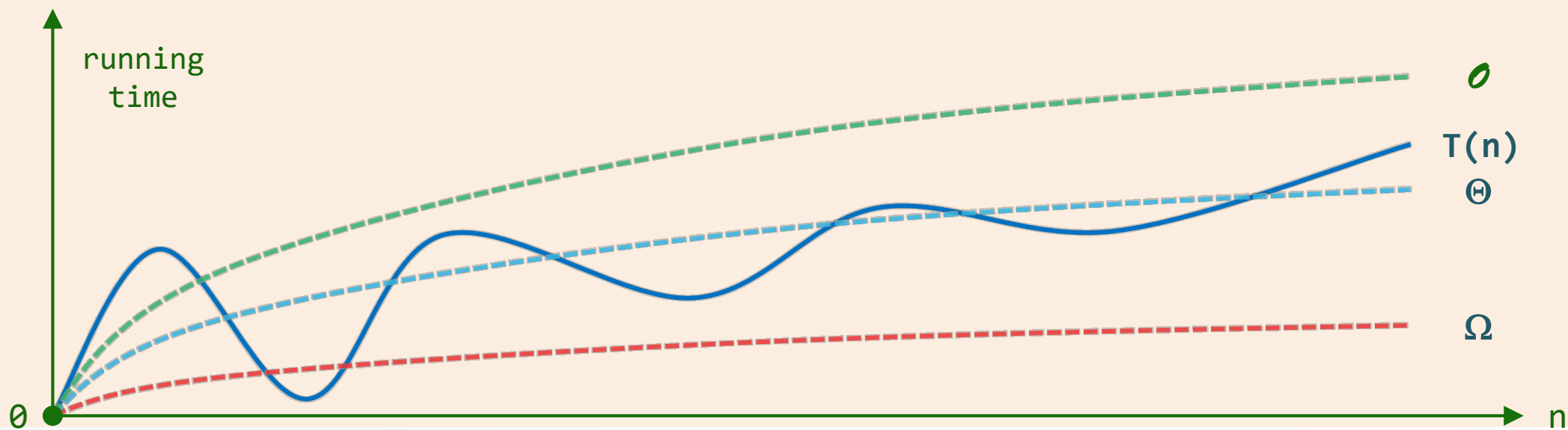
$$\mathcal{O}(n^a + n^b) = \mathcal{O}(n^a), \quad a \geq b > 0$$



其它记号

$$T(n) = \Omega(f(n)) \quad \text{iff} \quad \exists c > 0 \quad \text{s.t.} \quad T(n) > c \cdot f(n) \quad \forall n \gg 2$$

$$T(n) = \Theta(f(n)) \quad \text{iff} \quad \exists c_1 > c_2 > 0 \quad \text{s.t.} \quad c_1 \cdot f(n) > T(n) > c_2 \cdot f(n) \quad \forall n \gg 2$$



绪论

渐进复杂度：多项式

$O(1)$: constant

❖ 常数: $2 = 2021 = 2021 \times 2021 = O(1)$

//含RAM各基本操作, 甚至

$$2021^{2021} = O(1)$$

❖ 从渐近的角度来看, 再大的常数, 也要小于递增的变数

//尽管实际并非如此

❖ [General Twin Prime Conjecture, de Polignac 1849]

For every natural number k , there are infinitely many prime pairs p and q
such that $p - q = 2k$

❖ [Yitang Zhang, April 2013] $k \leq 35,000,000$

❖ [Terence Tao, May 2013] $k \leq 6,500,000$

❖ [Polymath Project, April 2014] $k \leq 123$

$\mathcal{O}(\log^c n)$: poly-log

❖ 对数 $\mathcal{O}(\log n)$: $\ln n$ $\lg n$ $\log_{100} n$ $\log_{2021} n$ //为何不注明底数?

❖ 常底数无所谓: $\forall a, b > 1, \log_a n = \boxed{\log_a b} \cdot \log_b n = \Theta(\log_b n)$

❖ 常数次幂无所谓: $\forall c > 0, \log n^c = c \cdot \log n = \Theta(\log n)$

❖ 对数多项式: $123 \cdot \log^{321} n + \log^{205} (7 \cdot n^2 - 15 \cdot n + 31) = \Theta(\log^{321} n)$

❖ 这类算法非常有效, 复杂度无限接近于常数: $\forall c > 0, \log n = \mathcal{O}(n^c)$

$\mathcal{O}(n^c)$: polynomial

❖ **多项式**: $a_k \cdot n^k + a_{k-1} \cdot n^{k-1} + \dots + a_2 \cdot n^2 + a_1 \cdot n + a_0 = \mathcal{O}(n^k), \quad a_k > 0$

$$100 \cdot n + 203 = \mathcal{O}(n) \quad \sqrt{23 \cdot n - 472} \times \sqrt{101 \cdot n + 203} = \mathcal{O}(n)$$

$$(100 \cdot n - 532) \cdot (20 \cdot n^2 - 445 \cdot n + 2021) = \mathcal{O}(n^3) \quad (2021 \cdot n^2 - 129) / (1991 \cdot n - 37) = \mathcal{O}(n)$$

$$\sqrt[3]{2 \cdot n^3 - \sqrt[3]{3 \cdot n^4 - \sqrt{4 \cdot n^5 + \sqrt{5 \cdot n^6 + \sqrt{6 \cdot n^7 + \sqrt{7 \cdot n^8 + \sqrt{8 \cdot n^9 + n^{2019} / \sqrt{n^6 - 5 \cdot n^3 + 1970}}}}} = \mathcal{O}(n^7)$$

❖ **线性 (linear function)** : 所有 $\mathcal{O}(n)$ 类函数

❖ 从 $\mathcal{O}(n)$ 到 $\mathcal{O}(n^2)$: 本课程编程习题主要覆盖的范围

❖ 这类算法的效率通常认为**已可**令人满意, 然而...这个标准是否**太低**了?

//P难度!

绪论

渐进复杂度：指数

$\mathcal{O}(2^n)$: exponential

❖ 指数: $T(n) = \mathcal{O}(a^n)$, $a > 1$

$$\because e^n = 1 + n + n^2/2! + n^3/3! + n^4/4! + \dots$$

$$\therefore \forall c > 1, n^c = \mathcal{O}(2^n)$$

$$n^{1000\dots 01} = \mathcal{O}(1.000\dots 01^n) = \mathcal{O}(2^n)$$

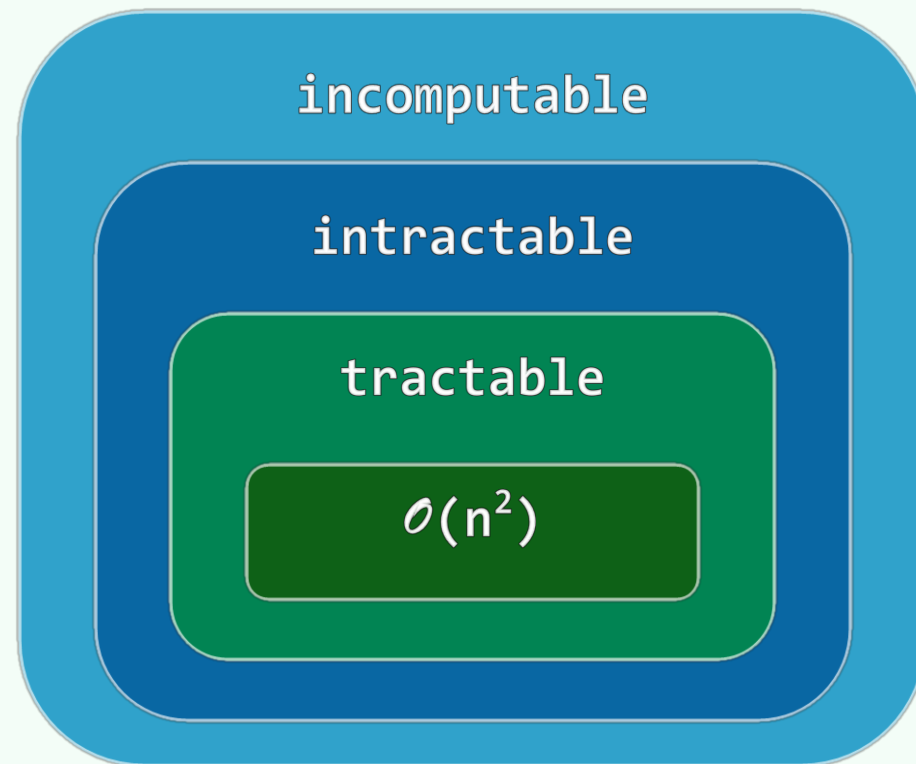
$$1.000\dots 01^n = \Omega(n^{1000\dots 01})$$

❖ 这类算法的计算成本增长极快，通常被认为不可忍受

❖ 从 $\mathcal{O}(n^c)$ 到 $\mathcal{O}(2^n)$ ，是从有效算法到无效算法的分水岭

❖ $\mathcal{O}(2^n)$ 算法往往显而易见，然而设计出 $\mathcal{O}(n^c)$ 算法却极其不易，有时甚至注定是徒劳无功

❖ 更糟糕的是，这类问题要远比我们想象的多得多...



2-Subset: 问题

$$\forall S = \{a_1, a_2, \dots, a_n\} \subset \mathbb{Z} \text{ and } \sum_{k=1}^n a_k = 2m$$

$$\exists T \subset S \text{ s.t. } \sum_{a \in T} a = m = \sum_{a \notin T} a ?$$

❖ 选举人团投票制:

50个州加1个特区, 共**538**票 //n, 2m

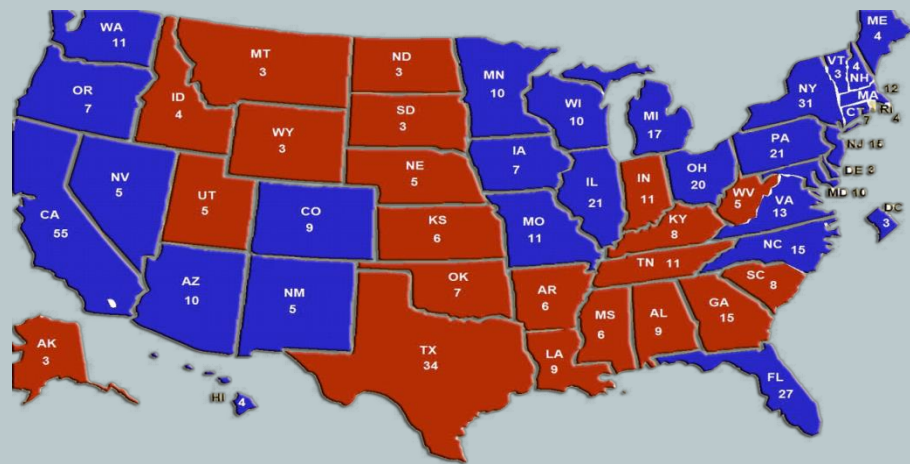
获**270**张选举人票, 即可当选 //m+1

❖ 若仅两位候选人, 会否恰好各得**269**票? //m

❖ 直觉上, 并**不难**:

逐一**枚举**s的每一子集, 统计总和并核对

55	California	11	Indiana	7	Connecticut	4	Idaho
34	Texas	11	Missouri	7	Iowa	4	Maine
31	New York	11	Tennessee	7	Oklahoma	4	New Hampshire
27	Florida	11	Washington	7	Oregon	4	Rhode Island
21	Illinois	10	Arizona	6	Arkansas	3	Alaska
21	Pennsylvania	10	Maryland	6	Kansas	3	Delaware
20	Ohio	10	Minnesota	6	Mississippi	3	D. C.
17	Michigan	10	Wisconsin	5	Nebraska	3	Montana
15	Georgia	9	Alabama	5	Nevada	3	North Dakota
15	New Jersey	9	Colorado	5	New Mexico	3	South Dakota
15	North Carolina	9	Louisiana	5	Utah	3	Vermont
13	Virginia	8	Kentucky	5	West Virginia	3	Wyoming
12	Massachusetts	8	South Carolina	4	Hawaii		538 = Σ



2-Subset: NPC

❖ 定理: $|2^S| = 2^{|S|} = 2^n$

❖ 亦即: 直觉算法需要迭代 2^n 轮, 并 (在最坏情况下) 至少需要花费这么多的时间

故严格地讲, 这仍只是**程序**, 而不是**算法**

❖ 还是直觉: 应该有更好的办法吧?

❖ 定理: 2-Subset is **NP-complete**

— 什么意思?

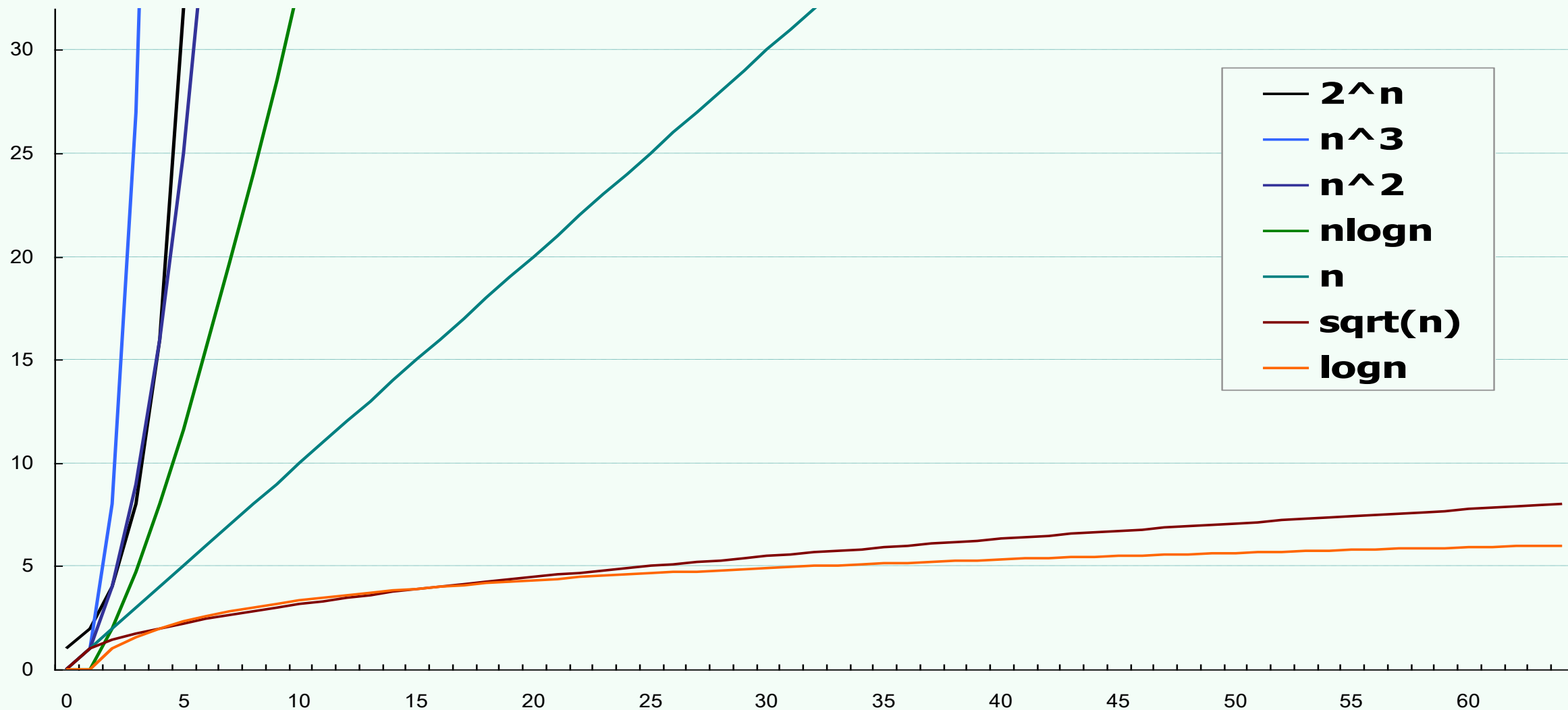
❖ 意即: 就目前的计算模型而言, **不存在**可在多项式时间内回答此问题的算法

— 就此意义而言, 上述的直觉算法已属最优

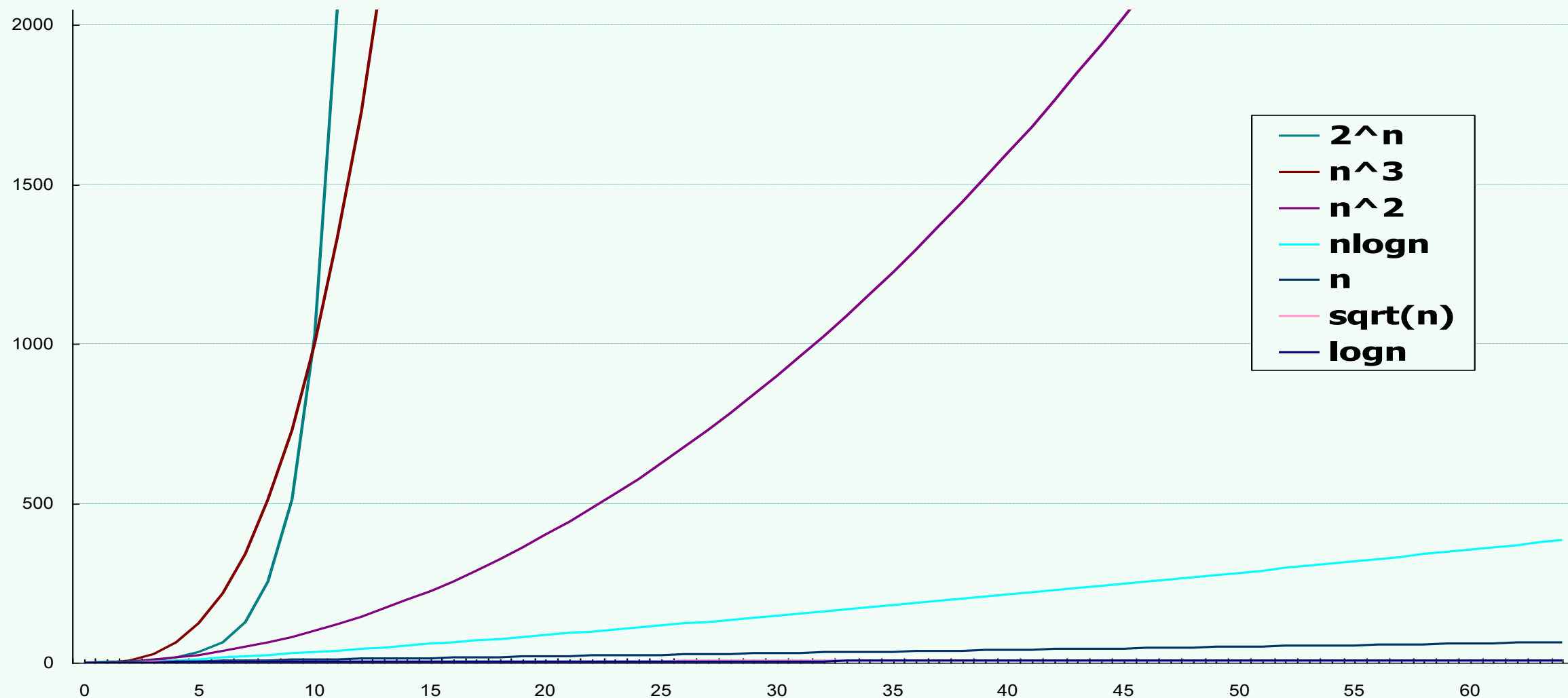
绪论

渐进复杂度： 层级划分

增长速度：先胖不算胖



增长速度：路遥知马力



层次级别

常数	$O(1)$	再好不过，但难得如此幸运	对数据结构的基本操作
	$O(\log^*n)$	在这个宇宙中，几乎就是常数	逆Ackermann函数
对数	$O(\log n)$	与常数无限接近，且不难遇到	有序向量的二分查找 堆、词典的查询、插入与删除
线性	$O(n)$	努力目标，经常遇到	树、图的遍历
	$O(n \log^*n)$	几乎几乎几乎...就是线性	某些MST算法
	$O(n \log \log n)$	非常非常非常...接近线性	某些三角剖分算法
	$O(n \log n)$	最常出现，但不见得最优	排序、EU、Huffman编码
平方	$O(n^2)$	所有输入对象两两组合	Dijkstra算法
立方	$O(n^3)$	不常见	矩阵乘法
多项式	$O(n^c)$	P问题 = 存在多项式算法的问题	
指数	$O(2^n)$	很多问题的平凡算法，再尽可能优化	
...		绝大多数问题，并不存在算法	

绪论

复杂度分析：级数

算法分析

- ❖ 两个主要任务 = 正确性 (不变性 x 单调性) + 复杂度
- ❖ 为确定后者, 真地需要将算法描述为RAM的基本指令, 再累计各条代码的执行次数? 不必!
- ❖ C++等高级语言的基本指令, 均等效于常数条RAM的基本指令; 在渐近意义下, 二者大体相当
 - 分支转向: `goto` //算法的灵魂; 为结构化而被隐藏了而已
 - 迭代循环: `for()`、`while()`、... //本质上就是 “if + goto”
 - 调用 + 递归 (自我调用) //本质上也是goto
- ❖ 主要方法: 迭代 (级数求和)、递归 (递归跟踪 + 递推方程)、实用 (猜测 + 验证)

级数

❖ **算术级数：与末项平方同阶** $T(n) = 1 + 2 + \dots + n = \frac{n(n+1)}{2} = \mathcal{O}(n^2)$

❖ **幂方级数：比幂次高出一阶** $\sum_{k=0}^n k^d \approx \int_0^n x^d dx = \frac{x^{d+1}}{d+1} \Big|_0^n = \frac{n^{d+1}}{d+1} = \mathcal{O}(n^{d+1})$

$$T_2(n) = \sum_{k=1}^n k^2 = 1^2 + 2^2 + 3^2 + \dots + n^2 = \frac{n(n+1)(2n+1)}{6} = \mathcal{O}(n^3)$$

$$T_3(n) = \sum_{k=1}^n k^3 = 1^3 + 2^3 + 3^3 + \dots + n^3 = \frac{n^2(n+1)^2}{4} = \mathcal{O}(n^4)$$

$$T_4(n) = \sum_{k=1}^n k^4 = 1^4 + 2^4 + 3^4 + \dots + n^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30} = \mathcal{O}(n^5)$$

❖ **几何级数：与末项同阶**

$$T_a(n) = \sum_{k=0}^n a^k = a^0 + a^1 + a^2 + a^3 + \dots + a^n = \frac{a^{n+1} - 1}{a - 1} = \mathcal{O}(a^n), \quad 1 < a$$

$$T_2(n) = \sum_{k=0}^n 2^k = 1 + 2 + 4 + 8 + \dots + 2^n = 2^{n+1} - 1 = \mathcal{O}(2^{n+1}) = \mathcal{O}(2^n)$$

收敛级数

$$\diamond \sum_{k=2}^n \frac{1}{(k-1) \cdot k} = \frac{1}{1 \cdot 2} + \frac{1}{2 \cdot 3} + \frac{1}{3 \cdot 4} + \dots + \frac{1}{(n-1) \cdot n} = 1 - \frac{1}{n} = \mathcal{O}(1)$$

$$\sum_{k=1}^n \frac{1}{k^2} = 1 + \frac{1}{2^2} + \frac{1}{3^2} + \dots + \frac{1}{n^2} < 1 + \frac{1}{2^2} + \frac{1}{3^2} + \dots = \frac{\pi^2}{6} = \mathcal{O}(1)$$

$$\sum_{k \text{ is a perfect power}} \frac{1}{k-1} = \frac{1}{3} + \frac{1}{7} + \frac{1}{8} + \frac{1}{15} + \frac{1}{24} + \frac{1}{26} + \frac{1}{31} + \frac{1}{35} + \dots = 1 = \mathcal{O}(1)$$

$$\diamond \text{几何分布: } (1 - \lambda) \cdot [1 + 2\lambda + 3\lambda^2 + 4\lambda^3 + \dots] = 1/(1 - \lambda) = \mathcal{O}(1), \quad 0 < \lambda < 1$$

❖ 有必要讨论这类级数吗？

难道，基本操作次数、存储单元数可能是分数？是的，某种意义上的是！

不收敛，但有限

❖ 调和级数: $h(n) = \sum_{k=1}^n \frac{1}{k} = 1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \dots + \frac{1}{n} = \ln n + \gamma + \mathcal{O}(\frac{1}{2n}) = \Theta(\log n)$

❖ 对数级数: $\sum_{k=1}^n \ln k = \ln \prod_{k=1}^n k = \ln n! \approx (n + 0.5) \cdot \ln n - n = \Theta(n \cdot \log n)$

❖ 对数 + 线性 + 指数: $\sum_{k=1}^n k \cdot \log k \approx \int_1^n x \ln x dx = \frac{x^2 \cdot (2 \cdot \ln x - 1)}{4} \Big|_1^n = \mathcal{O}(n^2 \log n)$

$$\begin{aligned} \sum_{k=1}^n k \cdot 2^k &= \sum_{k=1}^n k \cdot 2^{k+1} - \sum_{k=1}^n k \cdot 2^k = \sum_{k=1}^{n+1} (k-1) \cdot 2^k - \sum_{k=1}^n k \cdot 2^k \\ &= n \cdot 2^{n+1} - \sum_{k=1}^n 2^k = n \cdot 2^{n+1} - (2^{n+1} - 2) = (n-1) \cdot 2^{n+1} + 2 = \mathcal{O}(n \cdot 2^n) \end{aligned}$$

❖ 如有兴趣，不妨读读: Concrete Mathematics

//ex-2.35, Goldbach Theorem

绪论

复杂度分析：迭代

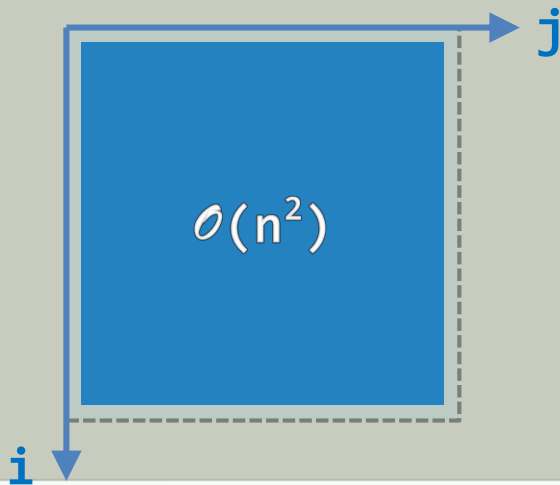
迭代 + 算术级数

```
for( int i = 0; i < n; i++ )
```

```
for( int j = 0; j < n; j++ )
```

```
    O1op(const i, const j);
```

$$\sum_{i=0}^{n-1} n = n \times n = \mathcal{O}(n^2)$$

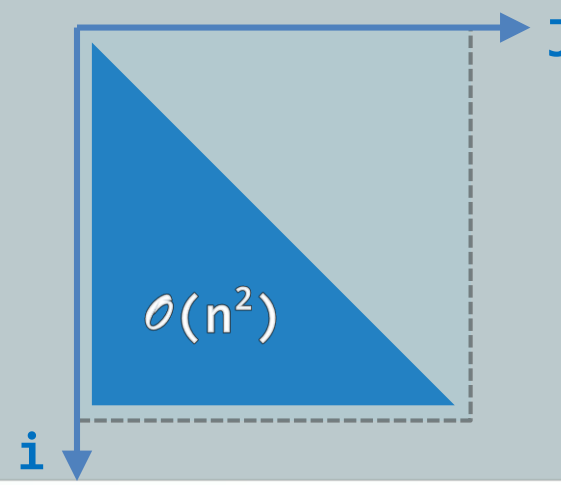


```
for( int i = 0; i < n; i++ )
```

```
for( int j = 0; j < i; j++ )
```

```
    O1op(const i, const j);
```

$$\sum_{i=0}^{n-1} i = \frac{n(n-1)}{2} = \mathcal{O}(n^2)$$

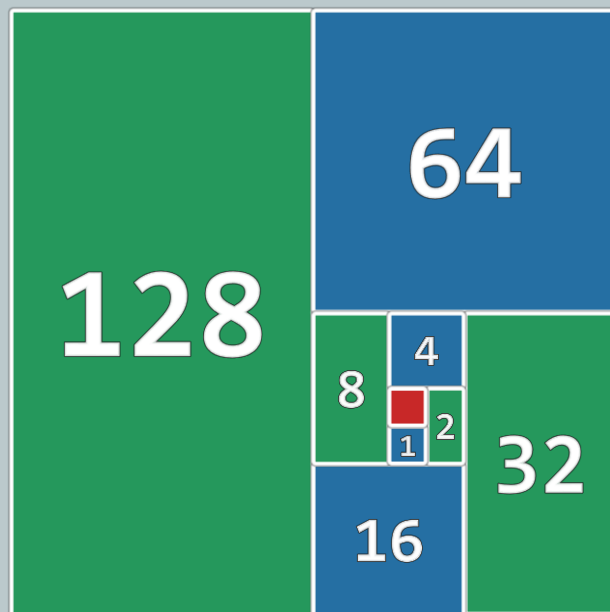


迭代 vs. 级数

```
for( int i = 1; i < n; i <<= 1 )
```

```
for( int j = 0; j < i; j++ )
```

```
    O1op( const i, const j );
```



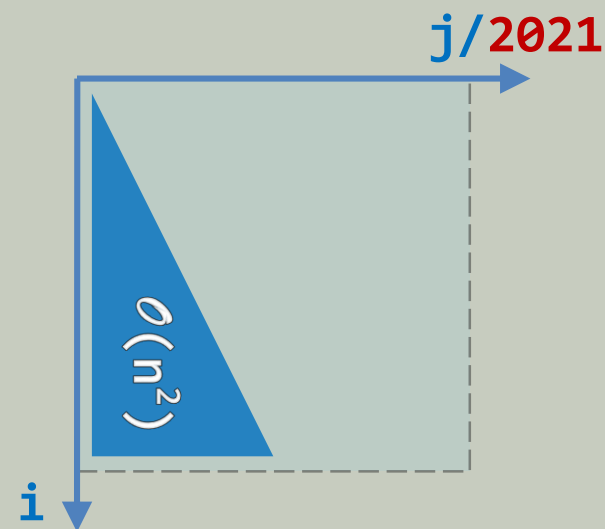
$$1 + 2 + 4 + \dots + 2^{\lfloor \log_2 (n-1) \rfloor}$$

$$= 2^{\lceil \log_2 n \rceil} - 1 = \mathcal{O}(n)$$

```
for( int i = 0; i < n; i++ )
```

```
for( int j = 0; j < i; j += 2021 )
```

```
    O1op( const i, const j );
```



绪论
封底估计

Back-Of-The-Envelope Calculation

❖ 地球 (赤道) 周长 $\approx 787 \times 360 / 7.2$

$$= 787 \times 50 = 39,350 \text{ km}$$

❖ 1天 = 24hr x 60min x 60sec

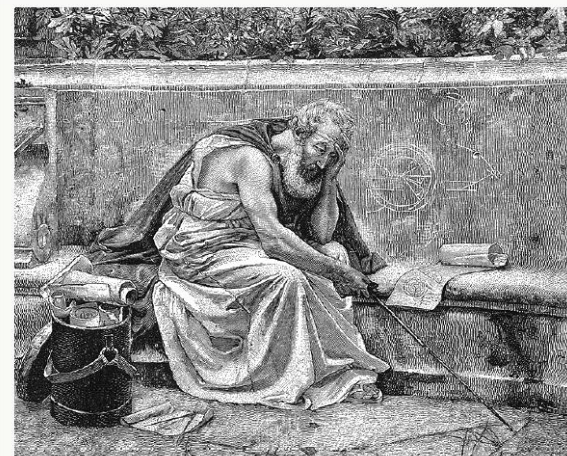
$$\approx 25 \times 4000 = 10^5 \text{ sec}$$

❖ 1生 $\approx$ 1世纪 = 100yr x 365 = $3 \times 10^4 \text{ day} = 3 \times 10^9 \text{ sec}$

❖ “为祖国健康工作五十年” $\approx 1.6 \times 10^9 \text{ sec}$

❖ “三生三世” $\approx 300 \text{ yr} = 10^{10} = (1 \text{ googol})^{(1/10)} \text{ sec}$

❖ 宇宙大爆炸至今 = $4 \times 10^{17} > 10^8 \times \text{一生}$



Eratosthenes
(276 ~ 194 B.C.)



Enrico Fermi
(1901 - 1954)

实例：全国人口普查数据的排序

$n = 1.4^+ \times 10^9$ $\mathcal{O}(10^9)$		Bubblesort $(10^9)^2$ 10^{18}	Mergesort $10^9 \times \log(10^9)$ 30×10^9	算法
硬件	普通PC 1GHz 10^9 flops	10^9 sec 30 yr	30 sec	
	天河1A 千万亿次 = 1P 10^{15} flops	10^3 sec 20 min	0.03 ms	

绪论

迭代与递归：减而治之

Sum

❖ 问题：计算任意 n 个整数之和

❖ `int SumI( int A[], int n ) {`

`int sum = 0; //O(1)`

`for ( int i = 0; i < n; i++ )//O(n)`

`sum += A[i]; //O(1)`

`return sum; //O(1)`

`}`

❖ 思路：逐一取出每个元素，累加之

❖ 无论 $A[]$ 内容如何，都有：

$$T(n) = 1 + n \cdot 1 + 1$$

$$= n + 2$$

$$= O(n)$$

$$= \Omega(n)$$

$$= \Theta(n)$$

❖ 空间呢？

Decrease-and-conquer



❖ 为求解一个大规模的问题，可以

- 将其划分为两个子问题：其一**平凡**，另一规模**缩减** //单调性
- 分别求解子问题；再由子问题的解，得到原问题的解

Linear Recursion: Trace

❖ `sum( int A[], int n )`

```
{ return n < 1 ? 0 : sum(A, n - 1) + A[n - 1]; }
```

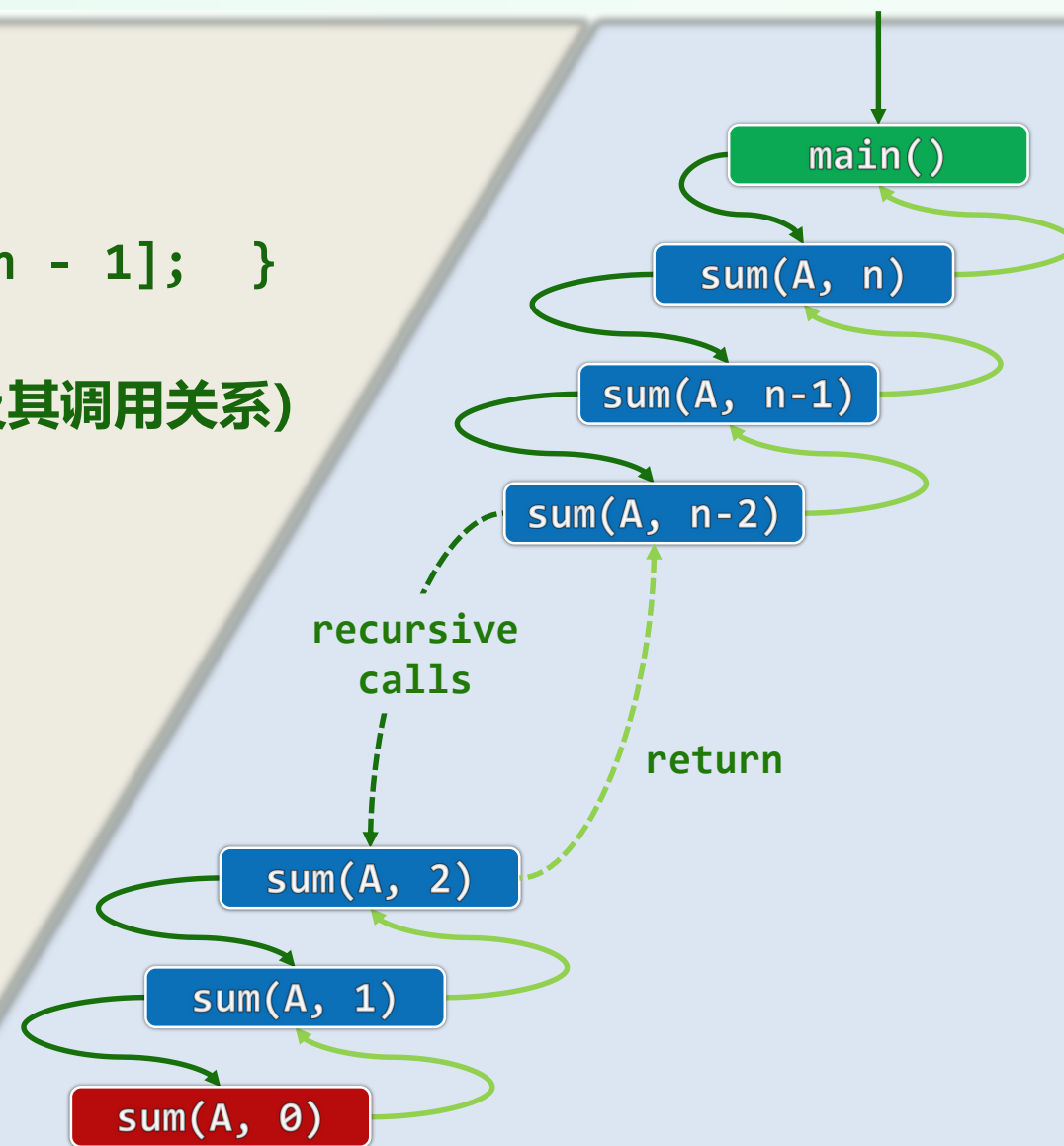
❖ 递归跟踪：绘出计算过程中出现过的所有**递归实例**（及其调用关系）

- 它们**各自**所需时间之总和，即为整体运行时间
- （**调用**操作本身的成本，如何处理？）

❖ 本例中，共计 $n+1$ 个递归实例，各自只需 $\mathcal{O}(1)$ 时间

故总体运行时间为： $T(n) = \mathcal{O}(1) \times (n + 1) = \mathcal{O}(n)$

❖ 空间复杂度呢？



Linear Recursion: Recurrence

❖ 对于大规模的问题、复杂的递归算法，递归跟踪不再适用

此时可采用另一抽象的方法...

❖ 从递推的角度看，为求解规模为 n 的问题 $\text{sum}(A, n)$ ，需 `//T(n)`

- 递归求解规模为 $n-1$ 的问题 $\text{sum}(A, n - 1)$ ，再 `//T(n-1)`

- 累加上 $A[n - 1]$ `//O(1)`

❖ 递推方程： $T(n) = T(n - 1) + \mathcal{O}(1)$ `//recurrence`

$T(0) = \mathcal{O}(1)$ `//base: sum(A, 0)`

❖ 求解： $T(n) = T(n - 2) + \mathcal{O}(2) = T(n - 3) + \mathcal{O}(3) = \dots = T(0) + \mathcal{O}(n) = \mathcal{O}(n)$

Reverse

❖ `void reverse( int * A, int lo, int hi );`

`//将数组中的区间A[lo,hi]前后颠倒`

❖ **减治:** $Rev(lo, hi) = [hi] + Rev(lo + 1, hi - 1) + [lo]$

❖ `if (lo < hi) { //递归版`

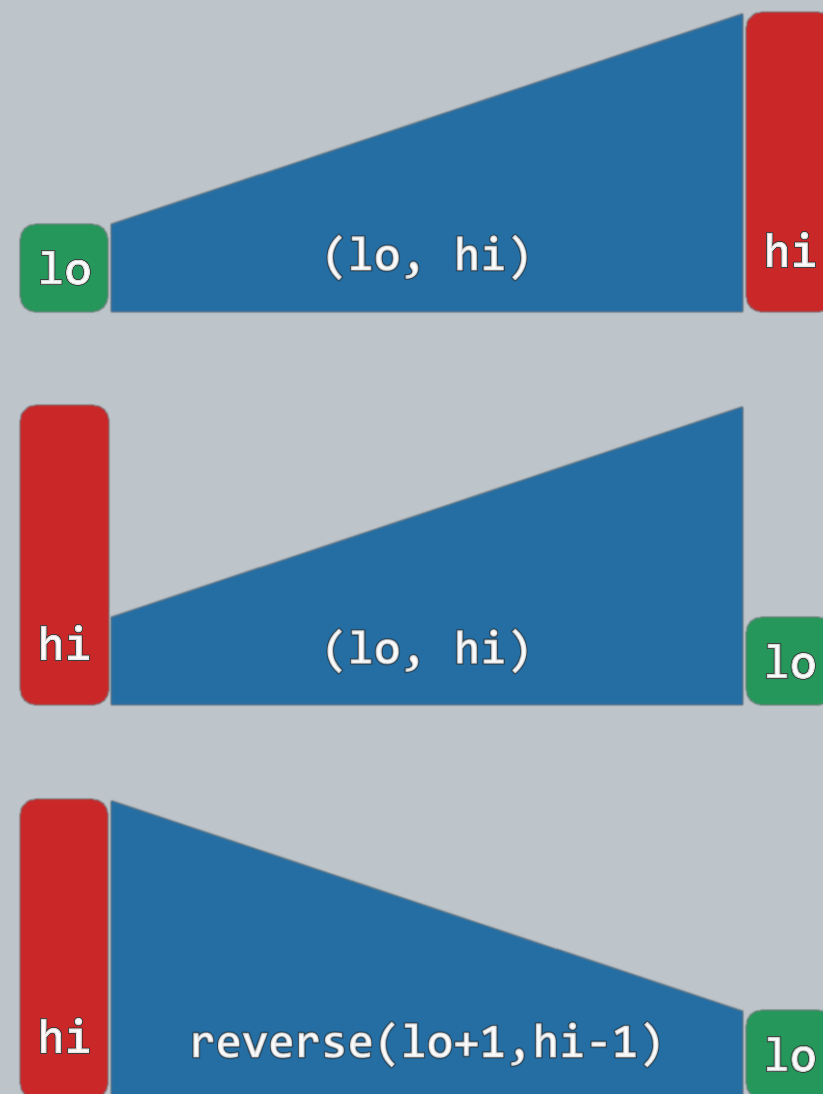
`swap( A[lo], A[hi] );`

`reverse( A, lo + 1, hi - 1 );`

`} //线性递归 (尾递归) ,  $O(n)$`

❖ `while (lo < hi) //迭代版`

`swap( A[lo++], A[hi--] ); //亦是 $O(n)$`



作业：

用递归追踪和递归方程的方法来求解reverse递归程序的时间复杂度。

绪论

迭代与递归：分而治之

Divide-and-Conquer

❖ 为求解一个大规模的问题，可以...

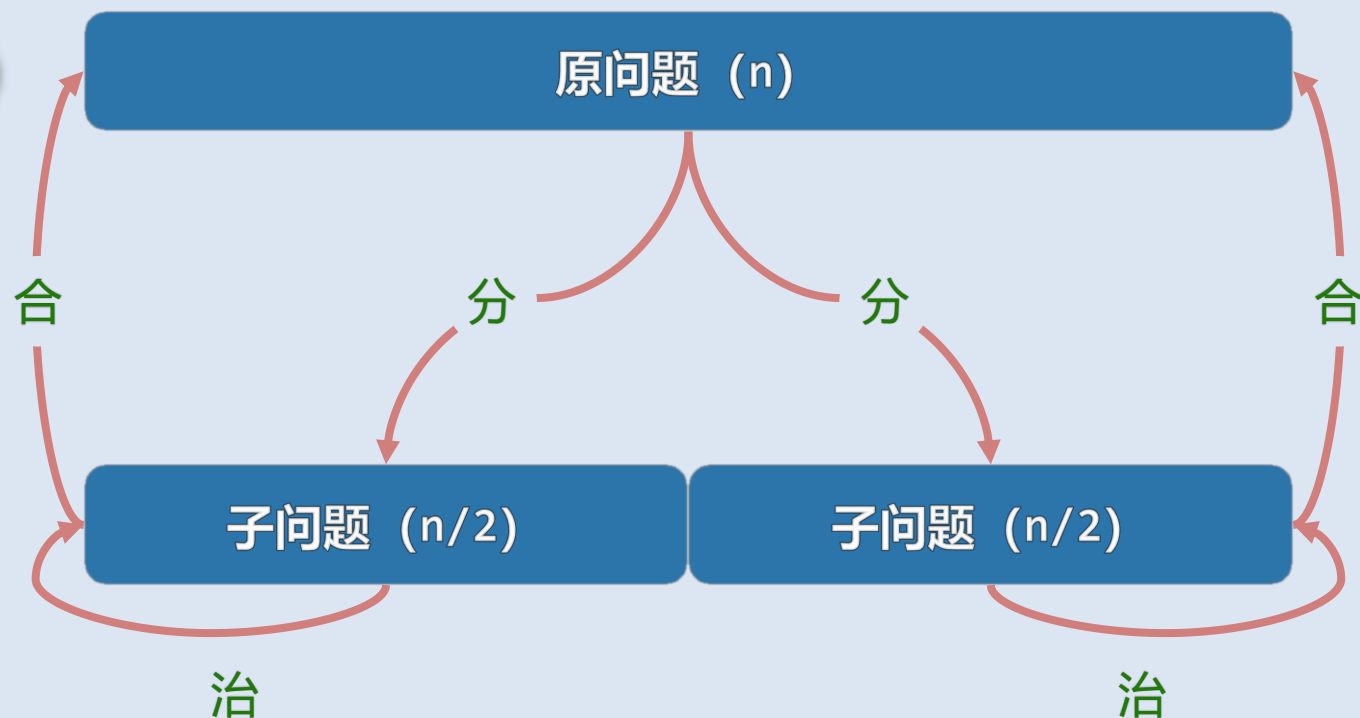
❖ 将其划分为若干子问题

(通常两个，且规模大体相当)

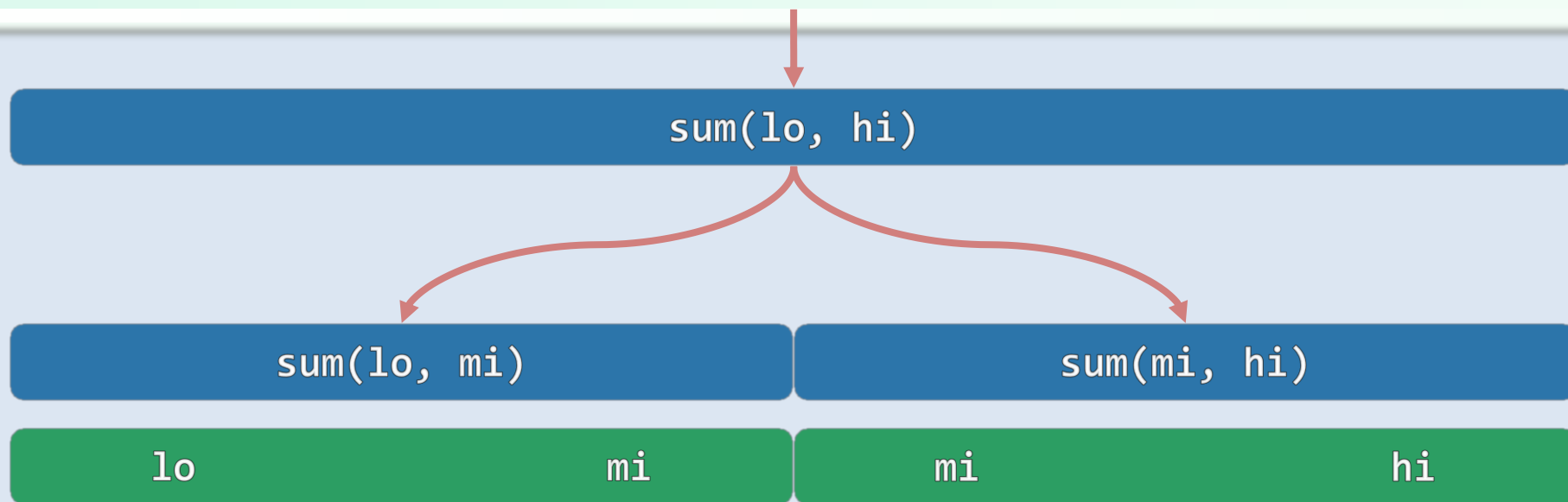
❖ 分别求解子问题

❖ 由子问题的解

合并得到原问题的解



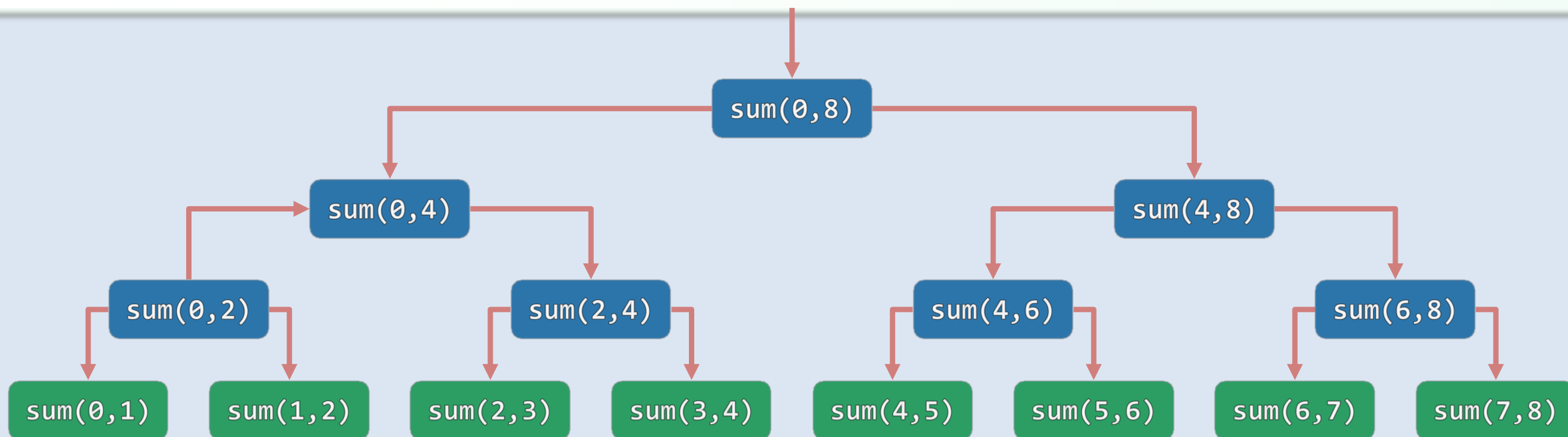
Binary Recursion



```
❖ sum( int A[], int lo, int hi ) { //区间范围A[lo, hi)
    if ( hi - lo < 2 ) return A[lo];

    int mi = (lo + hi) >> 1;    return sum( A, lo, mi ) + sum( A, mi, hi );
} //入口形式为sum( A, 0, n )
```

Binary Recursion: Trace



❖ $T(n)$ = 各层递归实例所需时间之和 //递归跟踪

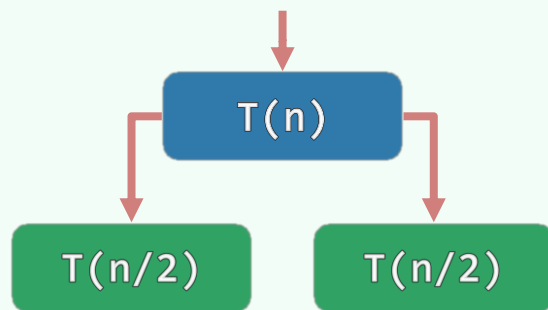
$$= \mathcal{O}(1) \times (2^0 + 2^1 + 2^2 + \dots + 2^{\log n})$$

$$= \mathcal{O}(1) \times (2^{1+\log n} - 1) = \mathcal{O}(n) \quad // \text{更快捷地, 作为几何级数...}$$

Binary Recursion: Recurrence

❖ 从递推的角度看，为求解 $\text{sum}(A, \text{lo}, \text{hi})$ ，需

- 递归求解 $\text{sum}(A, \text{lo}, \text{mi})$ 和 $\text{sum}(A, \text{mi}+1, \text{hi})$ ，进而 $//2 \cdot T(n/2)$
- 将子问题的解累加 $//O(1)$



❖ 递推方程: $T(n) = 2 \cdot T(n/2) + O(1)$

$$T(1) = O(1) \quad // \text{base: } \text{sum}(A, k, k)$$

❖ 求解:
$$\begin{aligned} T(n) &= 4 \cdot T(n/4) + O(3) = 8 \cdot T(n/8) + O(7) = 16 \cdot T(n/16) + O(15) = \dots \\ &= n \cdot T(1) + O(n-1) = O(2n-1) = O(n) \end{aligned}$$

绪论

动态规划：记忆法

fib(): 递归

$$fib(n) = fib(n-1) + fib(n-2)$$

0	1	1	2	3	5	8	13	21	34	55	89	...
---	---	---	---	---	---	---	----	----	----	----	----	-----

❖ `int fib(n) { return (2 > n) ? n : fib(n - 1) + fib(n - 2); }` //为何这么慢?

❖ 复杂度: $T(0) = T(1) = 1$; $T(n) = T(\boxed{n-1}) + T(\boxed{n-2}) + 1, \forall n > 1$

- 令 $S(n) = [T(n) + 1]/2$

- 则 $S(0) = 1 = fib(1), S(1) = 1 = fib(2)$

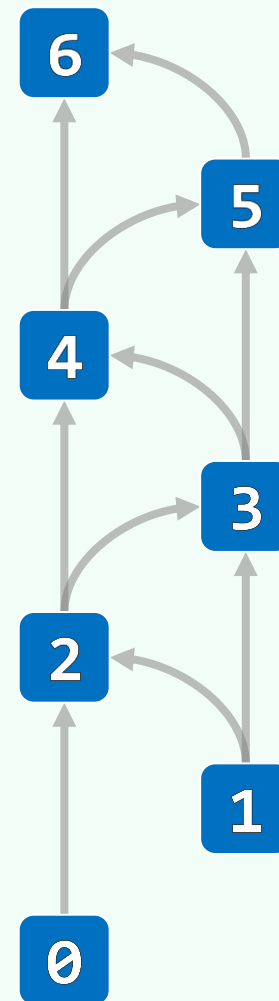
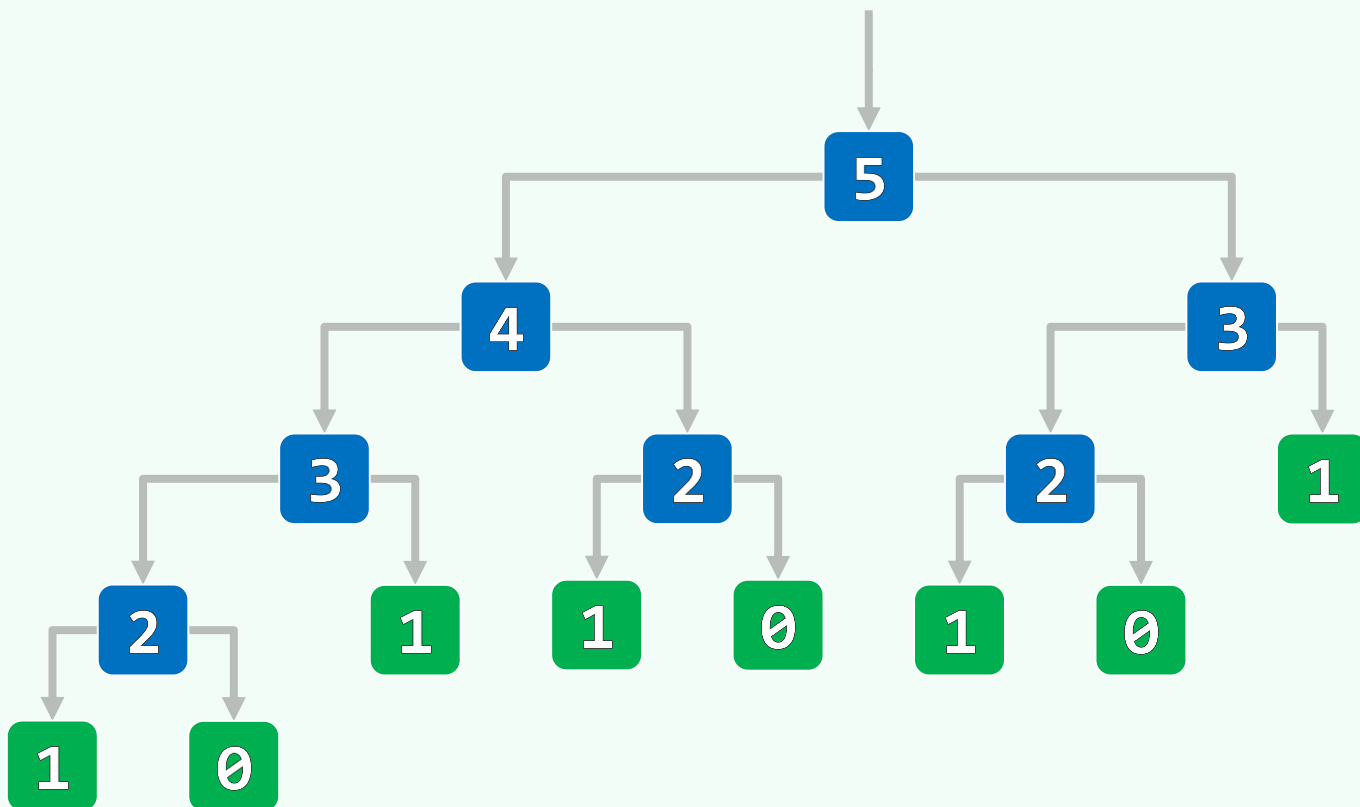
- 故 $S(n) = S(n-1) + S(n-2) = fib(n+1)$

$$T(n) = 2 \cdot S(n) - 1 = 2 \cdot fib(n+1) - 1 = \mathcal{O}(fib(n+1)) = \mathcal{O}(\phi^n)$$

- 其中 $\phi = (1 + \sqrt{5})/2 \approx 1.618$

递归

❖ 递归版fib()低效的根源在于，各递归实例均被大量地重复调用



❖ 先后出现的递归实例，共计 $\mathcal{O}(\phi^n)$ 个；而去除重复之后，总共不过 $\mathcal{O}(n)$ 种

Memoization



```
def f(n)
    if ( n < 1 ) return trivial( n );

    return f(n-X) + f(n-Y)*f(n-Z);
```

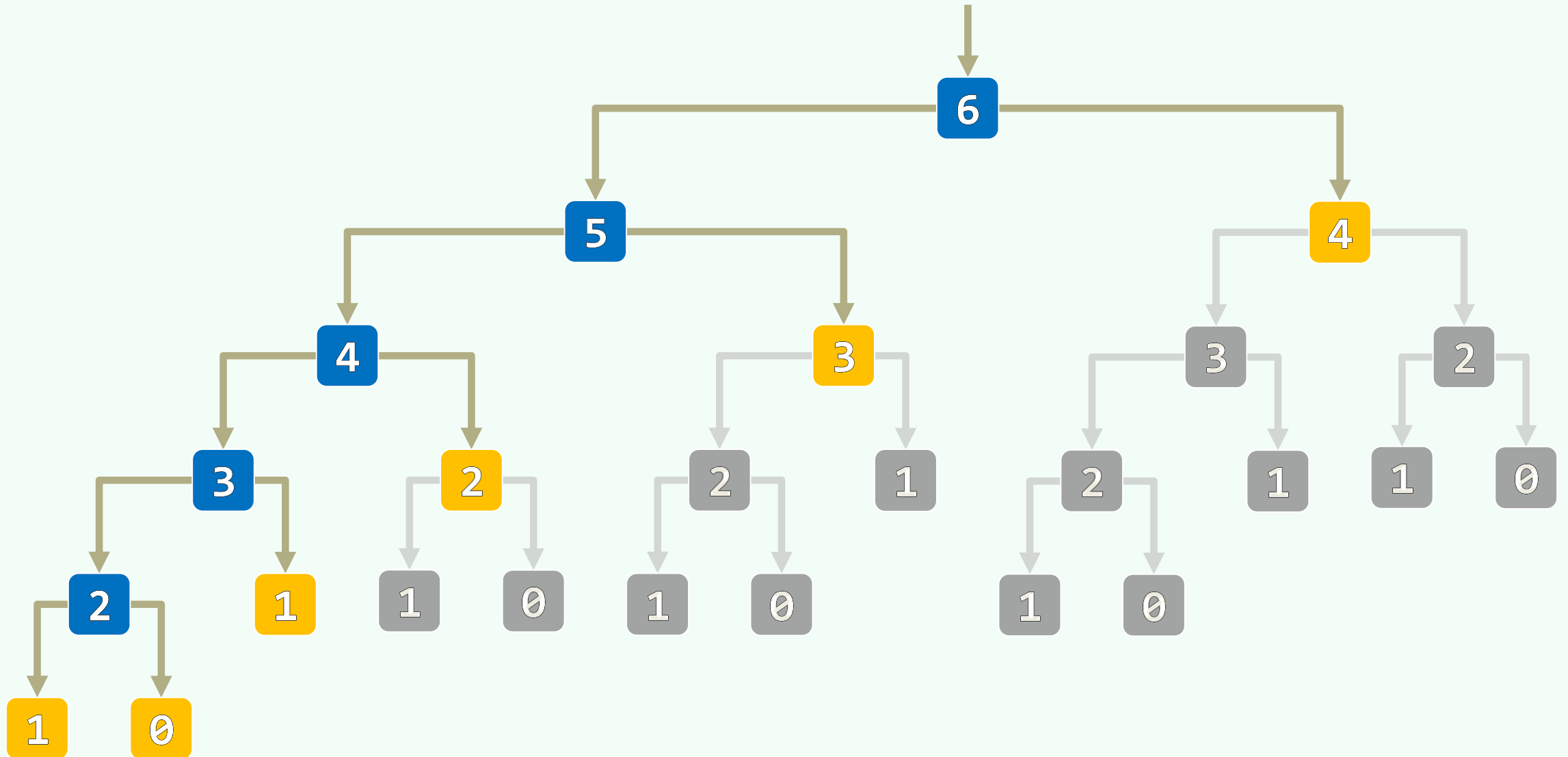
❖ T **M**[N]; #init. with UNDEFINED

```
def f(n)
    if ( n < 1 ) return trivial( n );

    # recur only when necessary &
    # always write down the result
    if ( M[n] == UNDEFINED )
        M[n] = f(n-X) + f(n-Y)*f(n-Z);

    return M[n];
```

Memoization: fib()

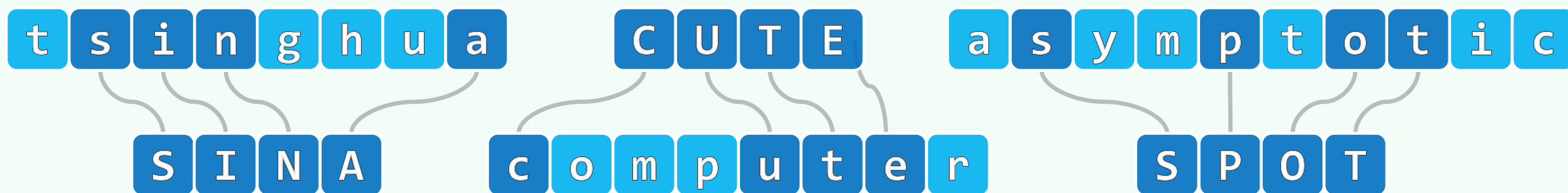


绪论

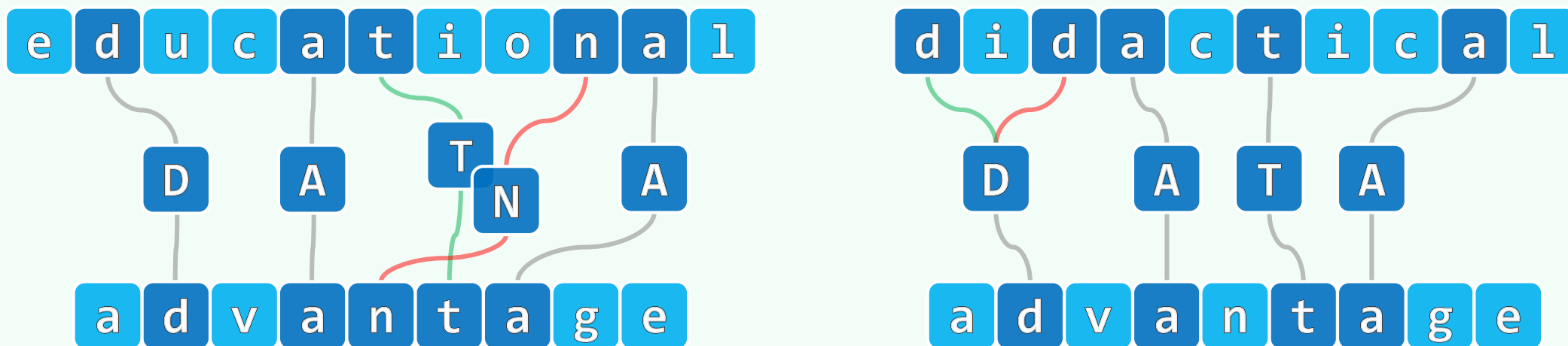
动态规划：最长公共子序列

问题

❖ 子序列 (Subsequence) : 由序列中若干字符, 按原相对次序构成



❖ 最长公共子序列 (Longest Common Subsequence) : 两个序列公共子序列中的最长者

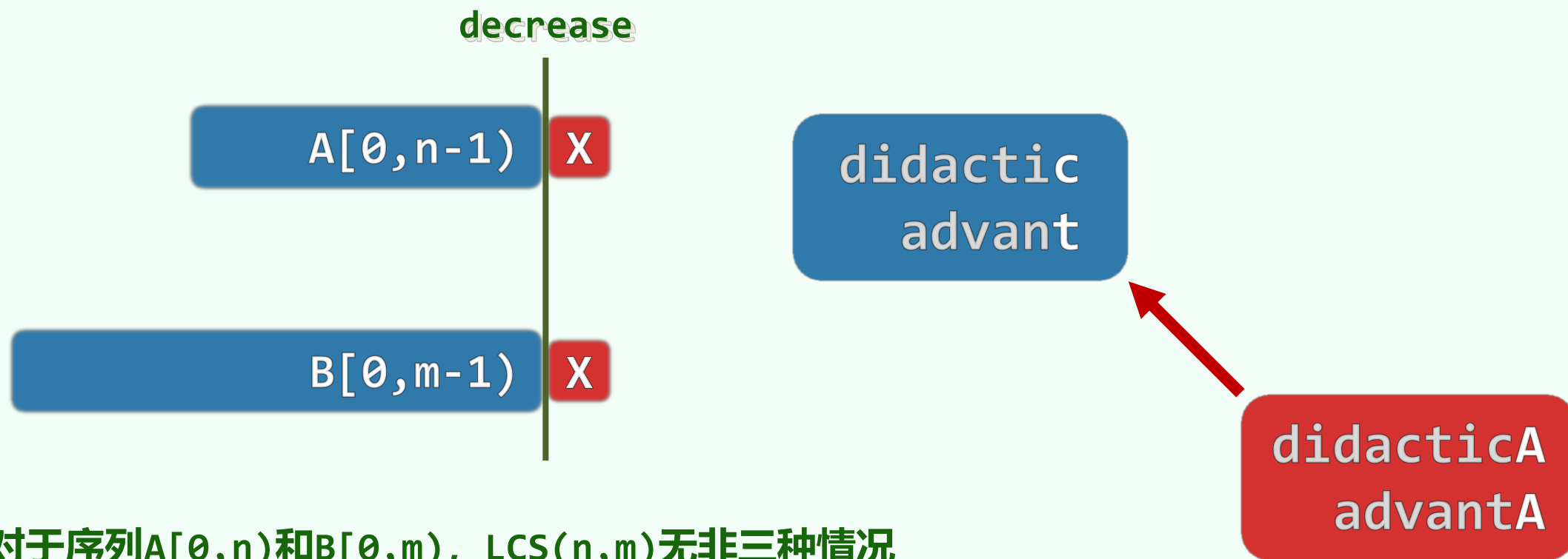


❖

可能有多个

可能有歧义

减治递归

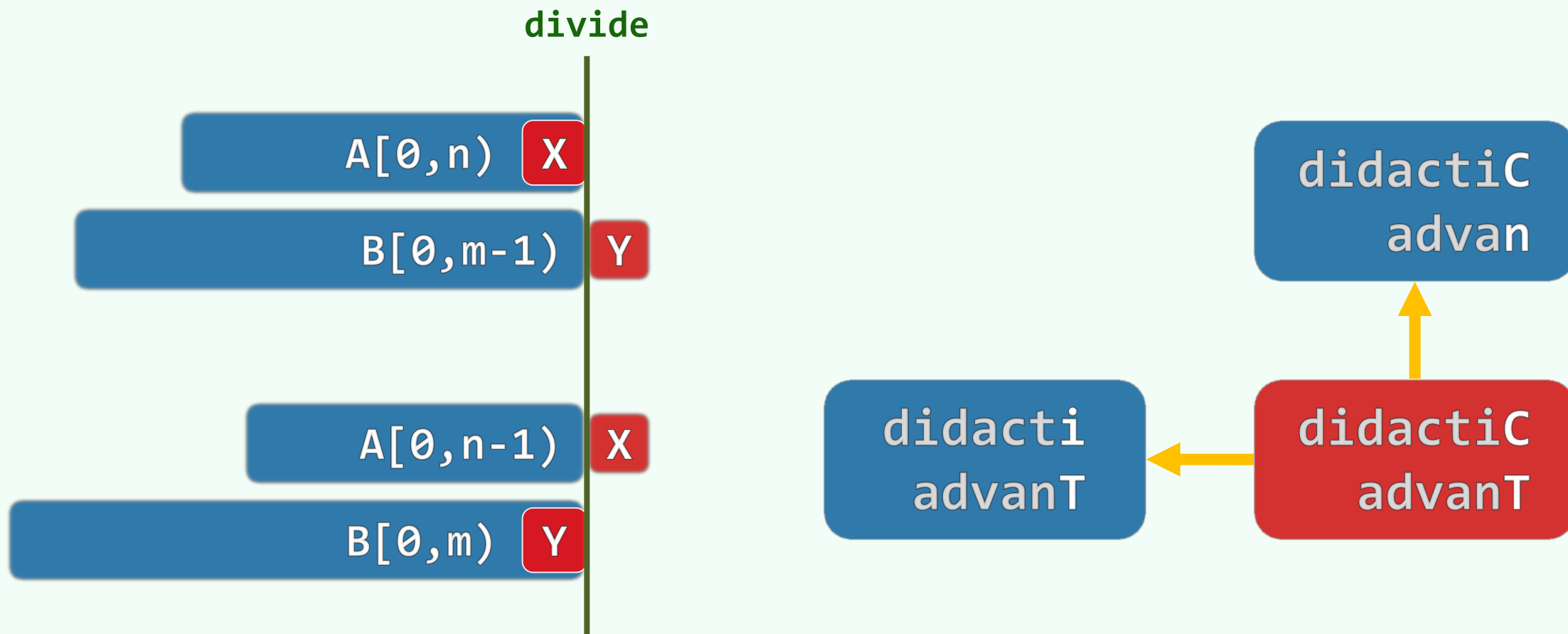


❖ 对于序列 $A[0, n)$ 和 $B[0, m)$, $LCS(n, m)$ 无非三种情况

0) 若 $n = 0$ 或 $m = 0$, 则取作空序列 ("") //递归基

1) 若 $A[n-1] = 'X' = B[m-1]$, 则取作: $LCS(n-1, m-1) + 'X'$ //减而治之

分治递归



2) $A[n-1] \neq B[m-1]$, 则在 $LCS(n, m-1)$ 与 $LCS(n-1, m)$ 中取更长者

//分而治之

描述：伪代码

Input: two strings A and B of length n and m resp.,

Output: (the length of) the longest common subsequence of A and B

`lcs( A[], n, B[], m )`

Compare the last character of A and B, i.e., `A[n-1]` and `B[m-1]`

If `A[n-1] = B[m-1]`

compute `x = lcs(A, n-1, B, m-1)` recursively and return `1 + x`

Else

compute `x = lcs(A, n-1, B, m)` and `y = lcs(A, n, B, m-1)` and return `max(x, y)`

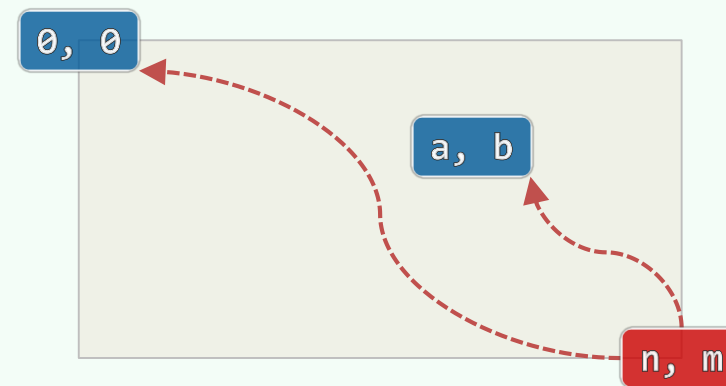
As the recursion base, return 0 when either n or m is 0

实现：递归版

```
unsigned int lcs(char const * A, int n, char const * B, int m) {  
  
    if (n < 1 || m < 1) //trial cases  
  
        return 0;  
  
    else if (A[n-1] == B[m-1]) //decrease & conquer  
  
        return 1 + lcs(A, n-1, B, m-1);  
  
    else //divide & conquer  
  
        return max(lcs(A, n-1, B, m), lcs(A, n, B, m-1));  
  
}
```

复杂度

- ❖ 单调性：每经一次比对，至少一个序列的长度缩短一个单位
- ❖ 最好情况，只需 $O(n + m)$ 时间 //比如...
- ❖ 然而最坏情况下，子问题数量不仅会增加，且可能大量雷同



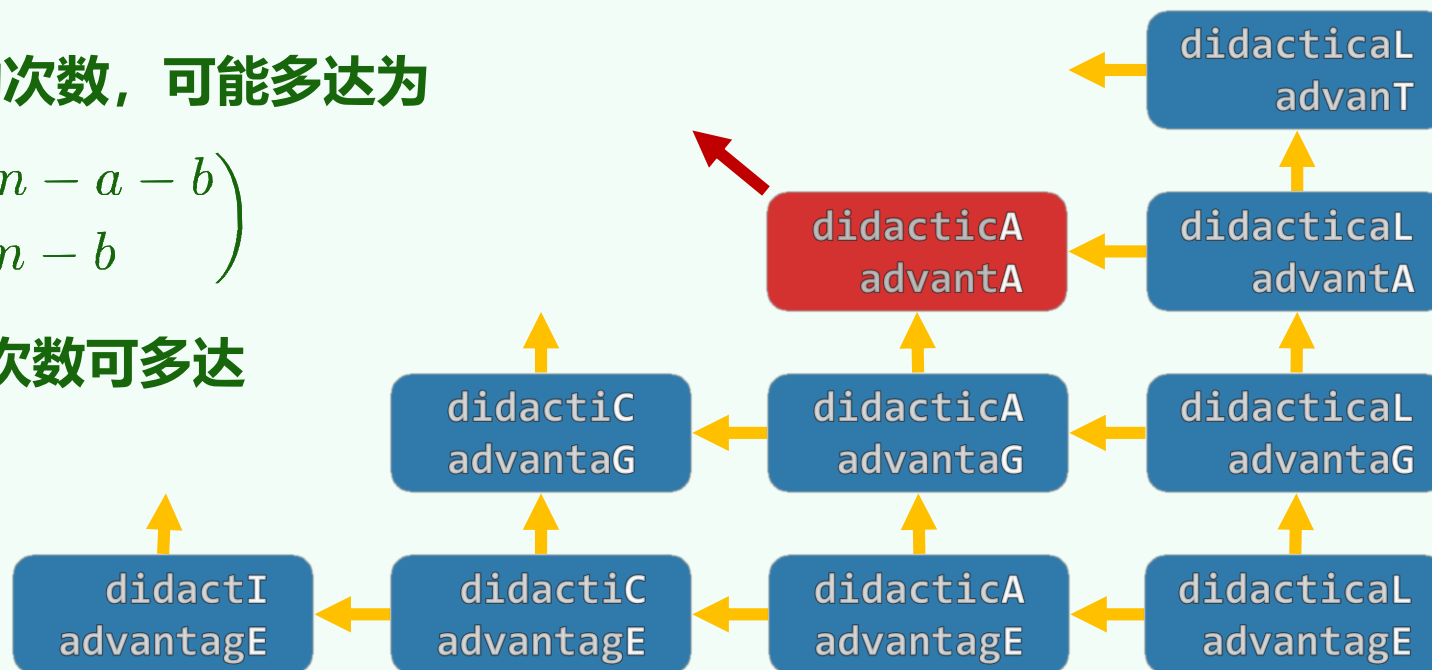
子任务 $LCS(A[a], B[b])$ 重复的次数，可能多达为

$$\binom{n + m - a - b}{n - a} = \binom{n + m - a - b}{m - b}$$

特别地， $LCS(A[0], B[0])$ 的次数可多达

$$\binom{n + m}{n} = \binom{n + m}{m}$$

当 $n = m$ 时，为 $\Omega(2^n)$



实现：记忆化版

```
unsigned int lcsM(char const * A, int n, char const * B, int m, unsigned int * const  
    if (n < 1 || m < 1) return 0; //trivial cases  
    if (UINT_MAX != lcs[(n-1)*M + m-1]) return lcs[(n-1)*M + m-1]; //recursion stops  
    else return lcs[(n-1)*M + m-1] = (A[n-1] == B[m-1]) ?  
        1 + lcsM(A, n-1, B, m-1, lcs, M)  
        max( lcsM(A, n-1, B, m, lcs, M), lcsM(A, n, B, m-1, lcs, M) );  
}  
  
unsigned int lcsMemo(char const* A, int n, char const* B, int m) {  
    unsigned int * lcs = new unsigned int[n*m]; //lookup-table of partial solutions  
    memset(lcs, 0xFF, sizeof(unsigned int)*n*m); //initialized with n*m UINT_MAX's  
    unsigned int solu = lcsM(A, n, B, m, lcs, m); delete[] lcs; return solu;  
}
```

动态规划

❖ 与fib()类似，这里也有大量重复的递归实例（子问题）

各子问题，分别对应于A和B的某个前缀组合

因此实际上，总共不过 $\mathcal{O}(n \cdot m)$ 种

❖ 采用动态规划的策略

只需 $\mathcal{O}(n \cdot m)$ 时间即可计算出所有子问题

❖ 为此，只需

- 将所有子问题（假想地）列成一张表

- 颠倒计算方向：从LCS(0,0)出发，依次计算出所有项——直至LCS(n,m)

		d	i	d	a	c	t	i	c	a	l
	0	0	0	0	0	0	0	0	0	0	0
a	0	0	0	0	1	1	1	1	1	1	1
d	0	1	1	1	1	1	1	1	1	1	1
v	0	1	1	1	1	1	1	1	1	1	1
a	0	1	1	1	2	2	2	2	2	2	2
n	0	1	1	1	2	2	2	2	2	2	2
t	0	1	1	1	2	2	3	3	3	3	3
a	0	1	1	1	2	2	3	3	3	4	4
g	0	1	1	1	2	2	3	3	3	4	4
e	0	1	1	1	2	2	3	3	3	4	4

实现：迭代（动态规划）版

```
unsigned int lcs(char const * A, int n, char const * B, int m) {  
    if (n < m) { swap(A, B); swap(n, m); } //make sure m <= n  
  
    unsigned int* lcs1 = new unsigned int[m+1]; //the current two rows are  
    unsigned int* lcs2 = new unsigned int[m+1]; //buffered alternatively  
  
    memset(lcs1, 0x00, sizeof(unsigned int) * (m+1));  
    memset(lcs2, 0x00, sizeof(unsigned int) * (m+1));  
  
    for (int i = 0; i < n; swap(lcs1, lcs2), i++)  
        for (int j = 0; j < m; j++)  
            lcs2[j+1] = (A[i] == B[j]) ? 1 + lcs1[j] : max(lcs2[j], lcs1[j+1]);  
  
    unsigned int solu = lcs1[m]; delete[] lcs1; delete[] lcs2; return solu;  
}
```